Documentation And Export

§ . . — Auto-generated docs sync mandate

Every auto-generated document (Issues_Summary, API reference, build manifest, etc.) MUST be regenerated in the **same commit** as any edit to its source. Stale auto-generated docs are §11.4 PASS-bluffs in document form: an operator reading them gets a divergent view of project state.

All three file types MUST stay in sync at all times — Markdown source + HTML export + PDF export (or equivalent multi-format output the project ships). Enforced by a pre-build gate that checks mtime ordering and content hash agreement.

§ . . — Script documentation mandate (User mandate, - -)

Forensic anchor — direct user mandate (verbatim, 2026-05-14):

> "Make sure that every single script inside the scripts dir (in all subdirs in depth) is fully documented and covered with full user manuals and user guides! ... Whenever is some bash script modified we MUST document it fully and create for it complete user guide(s) and manual(s) - if already exists make sure all of it is in sync and fully updated! No documentation ever can be out of sync with its codebase!"

Every Bash / shell / POSIX-sh script ANYWHERE in a project (anywhere under `scripts/`, `bin/`, `tests/`, library directories, `Upstreams/`, deployment hooks, CI helpers, etc. — depth-N recursive) MUST carry:

1. **In-source documentation block** at the top of the file:
   - Purpose: one-sentence description of what the script does.

    - Usage: complete invocation syntax with all flags, env vars, positional arguments, examples.
    - Inputs: env vars, files read, command-line args, stdin.
    - Outputs: files written, stdout/stderr behaviour, exit codes.
    - Side-effects: anything that changes system state outside the script's own scratch directory.
    - Dependencies: required commands (and how to install them) + required system state (e.g., "must be run as root", "must run from project root").
    - Cross-references: companion guides under `docs/`.
2. **External user guide** under `docs/scripts/<script-name>.md` (Markdown). Required sections:
    - **Overview** — what the script does in plain English.
    - **Prerequisites** — environment + dependencies.
    - **Usage examples** — copy-paste recipes for the common cases.
    - **Edge cases** — unusual invocations, error conditions, how to diagnose them.
    - **Internal behaviour** — for advanced users / future maintainers: what the script does step-by-step.
    - **Related scripts** — companion scripts and how they compose.
    - **Last verified version / date** — when the doc was last reconciled against the script source.

Whenever the script source is modified, the in-source documentation block AND the external user guide MUST be updated in the SAME commit. A commit that touches a script but leaves its documentation unchanged (or worse, lets it drift) is a §11.4.18 violation.

Pre-build gate `CM-SCRIPT-DOCS-SYNC` walks every `*.sh` and `*.bash` under `scripts/` (or equivalent script directories), verifies a companion `docs/scripts/<name>.md` exists, AND verifies the doc was modified in the same commit as the script (by SHA cross-reference, or by mtime ≥ script mtime as a softer floor). Paired mutation strips the doc-sync invariant and asserts the gate FAILs.

No documentation ever can be out of sync with its codebase.

This mandate composes with §11.4.11 (file-layout discipline — docs live under `docs/`, scripts live under `scripts/`) and §11.4.12 (auto-generated docs sync — the on-disk Markdown + its rendered HTML/PDF must all stay synchronised).

§ . . — Fixed-document column-alignment mandate (User mandate, - -)

Forensic anchor — direct user mandate (verbatim, 2026-05-14):

> "Make sure that Fixed document (all 3 file formats) is consistent and has the same structure and columns as Issues and Issues_Summary documents! This has to be added as detail to Constitution, CLAUDE.MD and AGENTS.MD (constitution Submodule where these information shall exist)."

Every project that maintains an Issues / open-work tracker AND a Fixed / closed-archive tracker MUST keep the two structurally aligned along the same lifecycle axes — at minimum **Status** and **Type** — so an operator reading either gets a consistent view across the entire lifecycle (open → in progress → ready for testing → in testing → reopened → fixed).

Specifically:

1. **Per-entry annotation.** Every heading in the Fixed archive document MUST carry, within 8 non-blank lines of the heading, a `**Status:**` line and a `**Type:**` line — exactly as §11.4.15 + §11.4.16 require for the Issues tracker. For Fixed entries the Status value is drawn from the **closure-set**:
   - `Fixed (→ Fixed.md)` — fully verified on device with captured evidence per §11.4.5.
   - `Fixed — pending device verification` — source-side fix landed, regression-protection gate wired, awaiting the next firmware build + flash + cycle for captured evidence.
   - `Fixed — RECLASSIFIED` — item was reclassified during investigation (test-side defect, environment-only, etc.).

   Type values follow §11.4.16: `Bug` | `Feature` | `Task`. Default on missing-annotation: `Task`.

2. **Auto-generated companion summary.** A `Fixed_Summary.md` MUST exist alongside `Fixed.md`, regenerated by an automation script (e.g. `generate_fixed_summary.sh`), and MUST mirror the column structure of `Issues_Summary.md` exactly:

   `| # | Level | Status | Type | One-line description |`

   The `Level` (severity) column uses the **original severity** the item had when it was open — so an operator can correlate a closed item with its original priority bucket. The same deterministic classification rules apply (REAL PRODUCT DEFECT / PARTIAL-or-WORKSTREAM-or-DEEP-DIVE / everything-else mapping to C / M / L).

3. **Three-format sync.** All three file types (`.md`, `.html`, `.pdf`) for BOTH the Fixed archive AND the Fixed_Summary MUST stay in sync at all times — same `.html` + `.pdf` export pipeline that §11.4.12 requires for Issues + Issues_Summary. The single-shot sync wrapper (e.g. `sync_issues_docs.sh`) MUST invoke the Fixed_Summary generator AND export all five doc variants (Issues, Issues_Summary, Fixed, Fixed_Summary, CONTINUATION) in one operation so they all carry the same mtime.

4. **Cross-lifecycle consistency.** A status line in Issues.md that reads `Fixed (→ Fixed.md)` is a **migration marker**, not a self-describing closure: when an Issues.md entry resolves, it MUST be moved to Fixed.md in the same commit, then disappear from Issues_Summary (because Issues_Summary only enumerates OPEN items) and appear in Fixed_Summary (because Fixed_Summary enumerates CLOSED items). Per §11.4.4 fix-closure protocol + §11.4.5 captured-evidence requirement, the migrated entry in Fixed.md MUST carry: closure cycle, closure commit SHA, captured evidence pointer, regression-protection gate name + paired mutation reference.

Pre-build gate `CM-FIXED-COLUMN-ALIGNMENT` (5+ invariants):

- `docs/Fixed_Summary.md` exists.
- `docs/Fixed_Summary.md` table header line contains both `Status` and `Type` columns (column-alignment with Issues_Summary).
- `mtime(Fixed_Summary.md) >= mtime(Fixed.md)` (regenerated after Fixed.md edits).

- Generator script exists (`generate_fixed_summary.sh` or equivalent integrated branch in `generate_issues_summary.sh`).
- Sync wrapper invokes the Fixed_Summary generator (grep for the script name in `sync_issues_docs.sh` or equivalent).
- HTML + PDF exports for both Fixed and Fixed_Summary exist (`docs/Fixed.html` , `docs/Fixed.pdf` , `docs/Fixed_Summary.html` , `docs/Fixed_Summary.pdf`).

Paired mutation: strip the Status column from `Fixed_Summary.md` table header (or rename the generator) → gate FAILs. Enforced by `meta_test_false_positive_proof.sh` .

Propagation. This anchor is a §11.4.17-classified **universal** rule — it composes naturally with §11.4.12 (auto-generated docs sync), §11.4.15 (Status tracking), §11.4.16 (Type tracking) and is mandatory across every consuming project's CLAUDE.md / AGENTS.md. The propagation gate `CM-COVENANT-114-19-PROPAGATION` (when implemented per consuming project) enforces the anchor's presence in every covenant file.

No escape hatch. A Fixed archive that lacks Status/Type columns or whose Summary companion drifts out of sync is the exact "different lifecycles report different facts" hazard §11.4 forbids — an operator reading Issues_Summary can no longer answer "where is item X now?" by cross-referencing Fixed_Summary because the schemas diverge. Non-compliance is a release blocker regardless of context.

§ . . — Document-sync commit discipline (User mandate, - -)

Forensic anchor — direct user mandate (verbatim, 2026-05-14):

> "For Issues, Issues_Summary and Fixed docs (and all its exports) we MUST commit and push (only them) as soon as they are updated (synced). Continuation doc and other similar / relevant documentation MUST be committed and pushed too! Like this we will always have up to date status of working items without need to do full commit and push of everything if that is not possible to do!"

Classification: §11.4.17-classified **universal** — every project that tracks work items through an Issues / Fixed lifecycle has the same problem, so the discipline lives in the Constitution and every consuming project implements its own wrapper.

The defect this anchor closes. Until this anchor existed, the only way to push a status update was the full-tree commit wrapper. When the working tree had unrelated heavy churn (large submodule diffs, in-flight rebase, partial-network conditions, ongoing build), operators had two bad options: (a) wait until the heavy work finishes — which can be hours — leaving the doc-status stale to every other agent; (b) skip the doc-status update entirely — which is a §11.4.4 documentation-drift violation. Neither option is acceptable.

The mandate. Every project under this Constitution that ships a status-tracking doc set MUST provide a **lightweight commit path** distinct from the full-repo commit wrapper. The lightweight path stages, commits, and pushes **only** the status-tracking doc set:

1. The active issue tracker (`docs/Issues.md` in a consuming project's layout; the project's equivalent file elsewhere) and its HTML + PDF exports.
2. The auto-generated issues summary (`docs/Issues_Summary.md`) and its HTML + PDF exports.
3. The fixed-bug archive (`docs/Fixed.md`) and its HTML + PDF exports.
4. The auto-generated fixed summary (`docs/Fixed_Summary.md`) and its HTML + PDF exports.
5. The cross-session continuation document (`docs/CONTINUATION.md` , per §12.10) and its HTML + PDF exports.
6. Any auto-generated audit artifact (`docs/audit/anti_bluff_audit.md` or equivalent) that pairs with the above.

The lightweight wrapper MUST:

1. **Auto-invoke the project's export-regeneration pipeline first** — so Markdown + HTML + PDF are all synchronised before push. (Skipped only when an explicit `--no-sync` flag is passed AND the operator just ran the pipeline manually.)
2. **Stage ONLY the doc-set files** — `git add` of an explicit file list, NEVER `git add -A` (which would catch unrelated WIP churn).
3. **Use a separate flock** disjoint from the full-tree commit wrapper's lock — so the two can run in parallel without contention on disjoint file sets.
4. **Push to every parent-repo remote** (reusing the project's push-all driver where available).
5. **Exit code semantics** — `0` success, `1` validation failure, `2` lock contention, `3` nothing-to-commit (informational, not an error — so the wrapper can be wired into automation that tolerates no-op outcomes).

The lightweight wrapper MUST be invocable either standalone OR via a flag on the full-tree wrapper (e.g. `commit_all.sh --docs-only`) so operators have a single mental model.

§9 preflight inheritance. The lightweight wrapper inherits the parent project's §9 preflight discipline — if `meta_test_*` is mid- mutation or `*.mut.bak` files exist, the wrapper refuses to run, same as the full-tree wrapper.

Pre-build gates (recommended, per consuming project):

- **`CM-COMMIT-DOCS-EXISTS`** — verifies the lightweight wrapper exists + is executable, its external user guide (per §11.4.18) exists, the full-tree wrapper advertises the delegation flag, and the wrapper's doc-set enumeration lists ≥ N entries (N depends on project — a consuming project's set is 15). Paired mutation strips the doc-set array → gate FAILs.

Propagation. Composes with §11.4.12 (export-sync invariant — HTML + PDF in sync with Markdown after every edit), §11.4.15 (item status tracking — Status lines must be visible quickly), §11.4.18 (every script ships with an in-source doc block + an external user guide), §12.10 (CONTINUATION document maintenance — the lightweight wrapper is one of the mechanisms that keeps CONTINUATION current).

No escape hatch. The discipline exists because doc-status drift is itself a §11.4 PASS-bluff pattern at the documentation layer: a green-looking `Issues_Summary.html` that doesn't reflect Markdown reality is the same class of

defect as a passing test that doesn't reflect runtime reality. Operators who can't run the lightweight wrapper for some reason MUST run the full-tree wrapper and accept the heavier cost — the doc-status drift is non-negotiable.

2026 05 14

§ . . — Visual-cue & grouping mandate for Issues docs (User mandate, - -)

Forensic anchor — direct user mandate (verbatim, 2026-05-14):

> "We MUST make sure that Issues, Issues_Summary and Fixed docs and its exported files (PDFs and HTMLs) have one small improvement: we MUST introduce some background coloring for cells of item type and status! Also, we MUST group items by the status! ... Base colors for types would be: bug - background pale red, task - background pale blue, feature - background pale yellow. However, changing the status affects the color of the cell of the type and the status cell background color: queued - no color, no effect, fixed - both cells pale green, in progress - only status cell is pale green, reopened - status cell is pale red, blocker - status cell is red (not pale, however MUST BE still readable), anything else please follow the same logic!"

Classification: §11.4.17-classified **mixed** — the discipline (visual cues + status grouping for tracked-item docs) is universal across every project that maintains an Issues / Fixed lifecycle. The implementation (CSS classes, HTML post-processor, weasyprint integration) is project-specific because each project's export toolchain differs (pandoc, asciidoctor, sphinx, etc.).

The defect this anchor closes. A long, multi-column tracked-item table where every row looks identical at first glance is a §11.4 operational-readability defect — operators have to *read* every Status column to identify what is queued vs in-progress vs fixed. With dozens to hundreds of items, this is slow enough that operators inevitably skim, and the doc effectively becomes write-only. Color cues + status grouping make at-a-glance triage possible.

The mandate. Every project under this Constitution that ships tracked-item docs (per §11.4.15) MUST apply visual-cue coloring AND status grouping to those docs' HTML + PDF exports. The Markdown source stays free of color noise (Markdown is the canonical text), but the HTML + PDF exports MUST be enriched in a post-processing stage:

Type cell base colors:

Type	Background
Bug	pale red (`#FFE5E5` or equivalent — ~5% red saturation)
Task	pale blue (`#E5F0FF`)
Feature	pale yellow (`#FFF8DC`)

Status cell colors (override the Status column; Fixed also overrides Type cell):

Status	Status cell	Effect on Type cell
Queued	no color (transparent)	keeps base type color
In progress	pale green (`#D4F1D4`)	keeps base type color
Ready for testing	pale green-blue (`#C6EAEF`)	keeps base type color
In testing	pale green-yellow (`#E7F4D6`)	keeps base type color
Reopened	pale red (`#FFCCCC`)	keeps base type color
Fixed	pale green (`#A8E6A8`)	**also** pale green
Operator-blocked / Blocker	vibrant red (`#FF4444`) with white text for readability	keeps base type color

Status grouping: items in the rendered table MUST be grouped by Status, emitting an H3-class heading for each Status group with the item count. Group order (most-actionable first; closed last):

1. Operator-blocked / Blocker
2. In testing
3. Ready for testing
4. In progress
5. Reopened
6. Queued
7. Fixed

This ordering surfaces the items requiring operator attention at the top of the doc; closed/archived items sink to the bottom.

Print-fidelity requirement. The CSS MUST declare `print-color-adjust: exact` (or the project's renderer's equivalent) so PDF exports preserve the cell backgrounds. A weasyprint default-rendered PDF that strips colors is a §11.4 export-drift defect — HTML and PDF MUST be visually equivalent.

Pre-build gates (recommended, per consuming project):

- **`CM-DOC-COLOR-GROUPING-DISCIPLINE`** — verifies the project's CSS carries the type+status class set, the post-processor exists and is executable, the sync wrapper invokes it, AND after sync the rendered HTML carries the grouping-heading + per-cell color classes (the captured-evidence proof — without this last invariant a stub post-processor could silently pass the static gate). Paired mutation strips a Status class literal from CSS → gate FAILs.

Propagation. Composes with §11.4.12 (export-sync invariant — HTML + PDF in sync with Markdown after every edit), §11.4.15 (item status tracking — every active item carries a Status line from the closed-set vocabulary), §11.4.19 (Fixed_Summary column-aligned companion), §11.4.22 (lightweight commit path for doc-set updates).

No escape hatch. Color cues are not optional polish — they are the operational-readability seam that converts a long tracked-item table from a write-only Markdown blob into a self-triaging document.

---

§ . . — Document Revision Header Mandate (User mandate, - -)

Forensic anchor — verbatim user mandate (2026-05-18):

> "Add this change / improvement as background work in parallel with our mainstream work. All documents (Issues, Issues_Summary, Fixed, Continuation doc and all others MUST HAVE revision number! Revision number with information such as: last date and time when document was modified MUST GO at the top of the document below the main H1 title. Add all relevant information about the document as well. However, last date and time document has been modified and revision number are MANDATORY! Revision number shall be the whole number 1, 2, 3, etc. Document all this in our root (constitution Submodule) Constitution, AGENTS.MD and CLAUDE.MD as mandatory rule(s) and constraints we MUST HAVE! No bluff is allowed of any kind!"

The mandate. Every IN-scope tracked Markdown document MUST carry a header block directly below the H1 title containing two MANDATORY fields:

- `**Revision:** N` where N is a monotonic positive integer (1, 2, 3, ...). Never decremented. Never reset on rewrite. A freshly- created IN-scope document starts at Revision 1.
- `**Last modified:** YYYY-MM-DDTHH:MM:SSZ` (ISO 8601 UTC).

Optional encouraged fields: `**Description:**`, `**Authority:**`, `**Maintainer:**`, `**Scope:**`, `**Auto-generated-from:**` (auto-generated docs only), `**Status:**` (plan docs only).

Header format (canonical):

```
# Document Title

**Revision:** 47
**Last modified:** 2026-05-18T13:42:00Z
**Description:** Open / in-flight bug + work tracker
**Authority:** Constitution §11.4 covenant
**Maintainer:** Operator + AI loop per §11.4.42

<content...>
```

YAML front-matter and HTML-comment headers are FORBIDDEN — they are invisible to humans scrolling the raw Markdown and break the "all relevant information about the document" visibility clause.

IN scope (revision header MANDATORY): `docs/Issues.md`, `docs/Issues_Summary.md`, `docs/Fixed.md`, `docs/Fixed_Summary.md`, `docs/CONTINUATION.md`, `docs/guides/**/*.md`, `docs/research/**/*.md`, `docs/scripts/**/*.md`, `docs/changelogs/**/*.md`, `docs/superpowers/plans/**/*.md`, `docs/hardware/**/*.md`, and every other tracked Markdown under `docs/`.

OUT scope (revision header NOT required): `CLAUDE.md` / `AGENTS.md` (every layer — already version-tracked via VERSION file + inheritance pointer per §11.4.35; adding a per-file revision would duplicate authority and obscure the canonical-root contract); `README.md` / `CONTRIBUTING.md` / `LICENSE` / `NOTICE` / `VERSION` / `OWNERS` (standard metadata); rendered HTML/PDF artifacts (revision inherited from source Markdown); auto- generated docs that already carry an embedded generation-time stamp.

Auto-bump tooling:

- `scripts/doc_revision_bump.sh <file>` — manual entry point, idempotent (no-op if Last-modified within 5 s of `date -u`).
- `.git/hooks/pre-commit` — automatic entry point; walks staged IN-scope docs, calls the bump for each, re-stages.
- `scripts/testing/sync_issues_docs.sh` — composes the bump for Issues_Summary + Fixed_Summary after regeneration.
- `scripts/commit_docs.sh` — calls the bump before staging so operators who skipped the hook installation are still covered.

Composition with §12.10. CONTINUATION.md already carries `Last updated:` per §12.10. §11.4.44 adds `**Revision:** N` directly under H1 + reuses the existing `Last updated:` line as the §11.4.44 `Last modified:` line. One source of truth, two pointers — no duplication.

Composition with §11.4.12. Issues_Summary.md / Fixed_Summary.md inherit the revision number of their source-of-truth Markdown at generation time; they are NOT incremented independently.

Composition with §11.4.23. HTML colorizer preserves the revision header in the colorized HTML — pandoc renders the bold lines as styled `<p><strong>...</strong></p>` blocks which the colorizer leaves alone (only Issues-table rows are mutated).

Gates: `CM-DOC-REVISION-HEADER-PRESENT` walks IN-scope docs and asserts both mandatory fields appear within 15 lines below the H1 title; `CM-COVENANT-114-44-PROPAGATION` asserts the anchor literal is present in every covenant file. Paired mutations strip the `**Revision:**` line OR the anchor literal → gates FAIL.

Classification: universal (per §11.4.17). **No escape hatch** — no `--skip-revision-bump`, `--no-header`, `--allow-missing-revision` flag exists. Operators who feel pressured to skip the bump should land smaller commits, not bypass the discipline.

2026 05 18 11 4 45

§ . . — Integration-Status-Doc Maintenance Mandate (User mandate, - -)

Forensic anchor — verbatim user mandate (2026-05-18):

> "Make sure this Markdown document is regularly updated to reflect current up to date state of full integration ... Whole integrations MUST be regularly updated and retested! Keep the Markdown document ALWAYS up to date and ALWAYS exported into PDF and HTML! Make sure document is ALWAYS in sync! Add all this into our Constitution, AGENTS.MD and CLAUDE.MD."

Generalisation note. §11.4.45 is the generic form of §12.10 (CONTINUATION.md) applied to every domain integration. §12.10 binds the canonical handoff document specifically; §11.4.45 generalises the sync-and-evidence pattern to every non-trivial integration (Dolby/Arvus, NanoKVM, Firebase, Widevine L1/L3, Play Protect, Netflix, Cuttlefish, Chromecast, WiFi chip, ES8388 codec, Sonos, AC-3, Google Stack, HiFi audio, TV apps, etc.). Without this generalisation each new integration re-invents the same sync wrapper, revision-header discipline, captured-evidence requirement, and operator-blocked surface — duplicating effort and risking drift.

Operative rule (10 sub-points — every Status.md MUST hold ALL).

Every integration-status document at `docs/<domain>/<integration>/Status.md`:

1. MUST exist when a domain integration is non-trivial (more than one fix / test / gate landing for that integration).
2. MUST carry the §11.4.44 revision header directly below the H1 title (Revision + Last modified + Description + Authority + Maintainer + Scope).
3. MUST be auto-synced (HTML + PDF) on every related-test-cycle completion AND every fix touching the integration. "Auto-synced" means the wrapper from sub-point 5 is invoked; manual regeneration is forbidden as the sole path because operators will forget.
4. MUST be auto-colorized per §11.4.23 — Status column cells colored by lifecycle state, Type cells colored by Type, operator-blocked rows visually distinct.
5. MUST have a sync wrapper invocable as `bash scripts/testing/sync_integration_status.sh <Status.md>` OR a per-integration thin shell that delegates to the generic wrapper. The thin shell exists so operators can type a short command per integration without remembering paths.
6. MUST live under `docs/<domain>/<integration>/` directory structure — consistent layout, discovery by glob remains O(1).
7. MUST include a captured-evidence-driven status table per §11.4.5 — every status claim cites the test log path, recording file path, or sink-probe report that backs it. Status claims without evidence are §11.4 PASS-bluffs.
8. MUST distinguish status values per §11.4.6 / §11.4.7: PASS / FAIL / SKIP / PENDING_FORENSICS / OPERATOR-BLOCKED — closed vocabulary, mechanically distinguishable, no implicit values.

9. MUST list operator-blocked items at the top of the document (just below the revision header) so an operator scanning the Status.md finds action items in O(1) — not buried 200 lines deep in a chronological evidence log.
10. MUST be referenced from `docs/CONTINUATION.md` §3 (Active work) when any item in the Status.md is non-terminal (Queued / In progress / Reopened / Operator-blocked) — composes with §12.10 so the canonical handoff document points at the integration-status doc rather than re-stating its contents.

Gates:

- `CM-COVENANT-114-45-PROPAGATION` — anchor text propagated to every CLAUDE.md / AGENTS.md across the covenant file set.
- `CM-AF-INTEGRATION-STATUS-DOCS` — discovers all `docs/**/Status.md` via glob, verifies each carries the §11.4.44 header, has a sync wrapper, HTML+PDF exports are mtime-current vs the source, colorization applied (cell-status class present in the colorized HTML).

Paired meta-test mutations (§1.1 compliance): propagation strip → propagation gate FAILs; delete `**Revision:**` from a Status.md → status-docs gate FAILs; touch Status.md without re-syncing → mtime mismatch FAILs; inject `likely` outside `UNCONFIRMED:` block → existing `CM-NO-GUESSING-MANDATE` catches it (composition catch).

Composition. §11.4.5 (captured evidence in every row), §11.4.6 / §11.4.7 (status vocabulary closed-set), §11.4.12 (sync wrapper pattern reused for HTML+PDF re-export), §11.4.13 (sink-side captured evidence is a specific instance of the generic rule), §11.4.15 (status values), §11.4.22 (commit_docs.sh wrapper), §11.4.23 (colorizer extends to Status.md HTML), §11.4.44 (every Status.md carries the revision header), §12.10 (CONTINUATION.md references Status.md paths in §3).

No escape hatch — no `--skip-status-sync`, `--no-revision-bump-on-status`, `--allow-stale-html` flag. Operators who feel pressured to skip the sync should land smaller, more frequent integration commits rather than batching status updates into one giant push.

Classification: universal (per §11.4.17).

§ . . — Fixed_Summary parity mandate (User mandate, - -)

Forensic anchor — verbatim user mandate (2026-05-18T17:55Z):

> "Note: Just like for Issues we have Issues_Summary, for Fixed we MUST HAVE Fixed_Summary - like all other docs: ALWAYS in sync and up to date and ALWAYS exported into the PDF and HTML! Add this mandatory rule / constraint into the root (constitution Submodule) Constitution, AGENTS.MD and CLAUDE.MD."

Why this anchor exists. §11.4.12 already established that `docs/Issues_Summary.md` is the canonical short-form summary of `docs/Issues.md` and MUST always be regenerated + re-exported (HTML

- PDF) whenever the source changes. §11.4.19 added the column- alignment requirement so `Fixed_Summary.md` mirrors `Issues_Summary.md` shape. §11.4.53 closes the propagation gap: the parity discipline that §11.4.12 imposes on Issues / Issues_Summary MUST apply symmetrically to Fixed / Fixed_Summary. The mechanical behaviour already exists in `scripts/testing/sync_issues_docs.sh` (lines 98- 105 regenerate Fixed_Summary in stage 1b alongside Issues_Summary in stage 1a), but the mandate was never explicit in the Constitution — and an unstated rule is a §11.4 PASS-bluff risk: future refactors of `sync_issues_docs.sh` could drop the Fixed_Summary half without violating any canonical authority. §11.4.53 makes the existing behaviour explicit canonical authority so the gate set can defend it.

Operative rule. `docs/Fixed_Summary.md` is the symmetric short- form summary of `docs/Fixed.md`. It MUST be regenerated whenever `Fixed.md` changes. Its HTML + PDF exports MUST travel with the markdown (identical mtimes within sync_issues_docs.sh granularity). Stale exports are §11.4.53 violations regardless of whether the underlying `.md` is correct — an operator (or future agent) reading the HTML or PDF gets a divergent view of which items are closed, and the §12.10 CONTINUATION resumption guarantee silently breaks. Same discipline as §11.4.12 Issues_Summary applied to Fixed.md.

Generator. `scripts/testing/generate_fixed_summary.sh` is the canonical generator. It MUST exist + be executable + emit a markdown table whose header columns include `Status` and `Type` (per §11.4.19 column-alignment) so the `Fixed_Summary.md` shape mirrors `Issues_Summary.md` exactly.

Auto-sync wrapper. `scripts/testing/sync_issues_docs.sh` is the single operator-facing entry point. It already regenerates BOTH Issues_Summary AND Fixed_Summary in one shot (stages 1a + 1b), then exports HTML + PDF (stage 2), then colorizes per §11.4.23 (stage 3), then re-renders the PDFs from the colorized HTML (stage 3b). MUST be invoked after any edit to `Fixed.md` (just as §11.4.12 requires after any edit to `Issues.md`). Manual invocation of just the Issues half (`--issues-only`-style flag) is FORBIDDEN — the wrapper has no such flag and §11.4.53 prohibits adding one.

Sort order. Same pattern as §11.4.12 — by closure date DESC (most-recent-Fixed first), with §-letter / Fix-# secondary sort. Documented at the top of the generated file.

HTML + PDF travel-together. Per §11.4.12 + §11.4.44, the three file types (`.md`, `.html`, `.pdf`) MUST always have identical mtimes within sync_issues_docs.sh granularity. Stale exports violate §11.4.53 regardless of whether the underlying `.md` is correct.

Composition:

- §11.4.12 — Issues_Summary parity is the sibling rule §11.4.53 symmetrizes. §11.4.12 and §11.4.53 are the canonical pair: edits to one source-of-truth (`Issues.md` or `Fixed.md`) trigger regeneration of BOTH summaries via the same wrapper.
- §11.4.19 — atomic Issues→Fixed migration triggers Fixed_Summary regeneration. When an item closes (status `Fixed (→ Fixed.md)` / `Implemented (→ Fixed.md)` / `Completed (→ Fixed.md)` per §11.4.33), the operator moves the entry from `Issues.md` to `Fixed.md` atomically and runs `sync_issues_docs.sh` — both summaries refresh.
- §11.4.23 — visual-cue & grouping colorizer post-processes both `Issues_Summary.html` AND `Fixed_Summary.html`. Stale `Fixed_Summary.html` would carry pre-§11.4.23 styling — itself a §11.4.53 violation.

- §11.4.33 — type-aware closure-status vocabulary. Fixed_Summary MUST respect the three terminal values (`Fixed (→ Fixed.md)` , `Implemented (→ Fixed.md)` , `Completed (→ Fixed.md)`) and emit them literally in the Status column.
- §11.4.44 — revision-header mandate applies to `Fixed_Summary.md` exactly as it applies to every other tracked Markdown doc.
- §12.10 — CONTINUATION.md resumption guarantee depends on the divergent-summary problem NOT existing; §11.4.53 is the explicit symmetric closure of that risk for the Fixed half.

Pre-build gates:

- `CM-FIXED-SUMMARY-SYNC` — canonical artifact-level gate (6 invariants): (1) `docs/Fixed_Summary.md` exists; (2) `docs/Fixed_Summary.html` exists with mtime ≥ `Fixed_Summary.md` mtime; (3) `docs/Fixed_Summary.pdf` exists with mtime ≥ `Fixed_Summary.md` mtime; (4) `Fixed_Summary.md` mtime ≥ `Fixed.md` mtime; (5) `scripts/testing/generate_fixed_summary.sh` exists + executable; (6) `scripts/testing/sync_issues_docs.sh` invokes the generator. Pre-build FAILs if any invariant is violated. Partially overlaps with `CM-FIXED-COLUMN-ALIGNMENT` (§11.4.19 Phase 39.AV gate) and `CM-DOCS-EXPORT-SYNC` (§11.4.15 4-doc + 6-export scan), but is explicitly the Fixed_Summary derived-doc parity invariant for §11.4.53 enforcement.
- `CM-COVENANT-114-53-PROPAGATION` — anchor literal `§11.4.53` present across all 5 canonical files (`constitution/Constitution.md` , `constitution/CLAUDE.md` , `constitution/AGENTS.md` , parent consumer's `CLAUDE.md` + `AGENTS.md`). Same propagation pattern as `CM-COVENANT-114-51-PROPAGATION` and `CM-COVENANT-114-52-PROPAGATION` . Consumer projects propagate further (owned submodules, nested submodules, HelixQA deps) at their own per-project gate granularity.

Paired mutations (per §1.1):

- Strip `§11.4.53` literal from `constitution/CLAUDE.md` → `CM-COVENANT-114-53-PROPAGATION` FAILs.
- Move `scripts/testing/generate_fixed_summary.sh` aside (so the invariant-(5) executable-check fails) → `CM-FIXED-SUMMARY-SYNC` FAILs.

- Touch `docs/Fixed_Summary.md` to a date older than `docs/Fixed.md` → invariant-(4) violated → `CM-FIXED-SUMMARY-SYNC` FAILs.

No escape hatch. No `--skip-fixed-summary-sync`, `--issues-only`, `--summary-not-applicable` flag exists. The discipline exists because the user mandate is unambiguous: "like all other docs: ALWAYS in sync and up to date and ALWAYS exported into the PDF and HTML". A divergent Fixed_Summary breaks operator + AI-agent ability to discover what is fixed vs what is still open — and that breakage IS the §11.4 "tests pass but reality differs" pattern at the documentation layer.

Classification: universal (per §11.4.17). Applies to every project consuming the constitution submodule that maintains an `Issues.md` / `Fixed.md` split (the Issues-tracking lifecycle is already a §11.4.15 universal mandate, so this clause inherits the same universal scope). Projects that do not maintain a Fixed.md (e.g., single-binary throwaway prototypes) are NOT_APPLICABLE; all projects that DO maintain a Fixed.md MUST land both gates and mutations within one cycle of adopting this Constitution version.

2026 05 19

§ . . — Status_Summary parity + two-audience format (User mandate, - -)

Forensic anchor — verbatim user mandate (2026-05-19):

> "Every Status.md doc gets a Status_Summary parity companion ALWAYS in sync, exported to HTML + PDF. Two-page format: page 1 = non-developer audience (team-specific: audio team for audio Status, video team for video Status, etc.), page 2 = software engineer summary. Auto-generated after every main Status update."

Why this anchor exists. §11.4.45 established `Status.md` per integration domain — but Status.md targets engineers (cites gate names, §-letter references, commit hashes, captured-evidence file paths). Non-developer stakeholders (audio team, video team, QA leads, product reviewers) need a plain-language version of the

same content, **always derived** from the engineering Status.md so the two views can never diverge. §11.4.56 introduces the symmetric companion doc with a two-audience layout.

Operative rule. For every `docs/<domain>/<integration>/ Status.md` that satisfies §11.4.45, a companion `docs/<domain>/<integration>/ Status_Summary.md` MUST exist with:

1. **Revision header** per §11.4.44.
2. **Page 1 — For the `<team>`** (audience-specific heading; the team name is derived from the integration domain — audio team for `docs/dolby/*`, video team for `docs/video/*`, etc.). Page 1 contains:
   - Plain-language summary of current state (1-3 paragraphs).
   - **What works** (1-3 bullets, end-user-visible behaviour).
   - **What's broken or pending** (1-3 bullets).
   - **Operator / team actions** if any.
   - NO code references, NO §-letter jargon, NO captured-evidence file paths, NO gate / mutation names.
3. **Page 2 — For software engineers** — same content density as Status.md but condensed:
   - §-letter references, gate names, commit hashes, captured- evidence file paths.
   - Cross-references to Issues.md / Fixed.md items by ATM-NNN (§11.4.54).
4. **HTML + PDF exports** travel with the markdown (identical mtimes within sync wrapper granularity).
5. **Auto-generation** — the canonical helper `scripts/testing/generate_status_summary.sh <Status.md path>` produces both pages. The generator MUST be invoked from a sync wrapper (e.g. `scripts/testing/sync_integration_status.sh` per §11.4.45) every time the source Status.md is updated.

Page-1 audience derivation. The generator maps domain → audience name via a project-controlled mapping table (e.g. `audio` → `Audio team`, `video` → `Video team`, `network` → `Network team`, `drm` → `DRM / streaming team`).

Unmapped domains default to `Project team`. The mapping table lives in the project's generator config, NOT in the Constitution (project-specific data per §11.4.17).

Composition:

- §11.4.45 (Status.md per integration) — Status_Summary.md COMPLEMENTS, never replaces. Both files exist together.
- §11.4.12 + §11.4.53 (parity discipline) — Status_Summary follows the same regeneration / HTML+PDF / mtime-alignment pattern.
- §11.4.44 (revision header) — applies to Status_Summary.md.
- §11.4.23 (colorizer) — Status_Summary.html receives the same type/status visual cues if it embeds tracked-item references.
- §12.10 (CONTINUATION.md) — non-developer stakeholders read Status_Summary; engineers read Status.md + CONTINUATION.md + Issues.md.

Pre-build gates:

- `CM-STATUS-SUMMARY-EXISTS-FOR-EVERY-STATUS` — for every `docs/**/Status.md`, the sibling `Status_Summary.md` MUST exist (+ HTML + PDF) with mtime ≥ Status.md mtime − tolerance.
- `CM-STATUS-SUMMARY-TWO-AUDIENCE` — every Status_Summary.md contains BOTH the `Page 1 — For the <team>` heading AND the `Page 2 — For software engineers` heading. Missing either FAILs.
- `CM-STATUS-SUMMARY-REVISION-HEADER` — §11.4.44 header present.
- `CM-COVENANT-114-56-PROPAGATION` — anchor literal across files.

Paired mutations (per §1.1):

- Delete a Status_Summary.md → `CM-STATUS-SUMMARY-EXISTS-FOR-EVERY-STATUS` FAILs.
- Remove `Page 1` heading from a Status_Summary.md → `CM-STATUS-SUMMARY-TWO-AUDIENCE` FAILs.
- Strip `§11.4.56` literal from `constitution/CLAUDE.md` → `CM-COVENANT-114-56-PROPAGATION` FAILs.

No escape hatch. No `--skip-status-summary`, `--engineer-only`, `--no-audience-split` flag. Plain-language stakeholder access is not negotiable — the project ships to humans, not just to engineers.

Classification: universal (per §11.4.17). Applies wherever §11.4.45 applies (every project that maintains domain-scoped Status.md docs).

2026 05 19

§ . . — README.md doc-link section + revision metadata (User mandate, - -)

Forensic anchor — verbatim user mandate (2026-05-19):

> "Add a doc-link section to README.md — links to Issues + Issues_Summary + Fixed + Fixed_Summary + CONTINUATION + ALL Status docs + their exports. Each link shows revision + last-modified."

Why this anchor exists. Operators, AI agents, and external reviewers arriving at the project repo for the first time need a single discoverable entry point listing every canonical tracked- items + status document, along with each document's freshness (revision + last-modified). Without this surface, agents resort to `find docs -name '*.md'` which returns hundreds of files unranked. §11.4.57 makes the README.md the canonical index.

Operative rule. Every project's top-level `README.md` MUST contain a section titled `Tracked-Items + Status Documents` (or the project's localized equivalent — the section heading MUST contain the literal `Tracked-Items` for gate detection). The section MUST be a markdown table with columns: `Document` (human- readable name), `Last modified` (ISO 8601 UTC from the doc's §11.4.44 revision header), `Revision` (integer from same header), `Markdown` (relative link to `.md`), `HTML` (relative link to `.html`), `PDF` (relative link to `.pdf`).

The section MUST link to:

1. `Issues.md`, `Issues_Summary.md` (§11.4.12, §11.4.15, §11.4.16).
2. `Fixed.md`, `Fixed_Summary.md` (§11.4.19, §11.4.53).

3. `CONTINUATION.md` (§12.10).
4. **Every** `docs/**/Status.md` and its `Status_Summary.md` pair (§11.4.45, §11.4.56). Status docs are auto-discovered by the generator via `find docs -name 'Status.md'`.

Generator. `scripts/testing/update_readme_doc_links.sh` is the canonical helper. It:

1. Scans the canonical doc paths.
2. Extracts each doc's `Revision` + `Last modified` fields from its §11.4.44 header.
3. Renders the markdown table with current values.
4. Replaces the section between explicit markers (`<!-- doc-link-section:begin -->` and `<!-- doc-link-section:end -->`) in README.md.

The wrapper MUST be invoked from `sync_issues_docs.sh` (so every Issues.md / Fixed.md update refreshes the README freshness data) AND from `sync_integration_status.sh` (so every Status.md update likewise refreshes README).

Composition:

- §11.4.12 + §11.4.19 + §11.4.53 (Issues / Issues_Summary / Fixed / Fixed_Summary parity) — README links to all four.
- §11.4.44 (revision header) — README pulls Revision + Last modified from each doc's header.
- §11.4.45 (Status.md) + §11.4.56 (Status_Summary.md) — README enumerates every Status pair.
- §12.10 (CONTINUATION.md) — explicitly linked from README so operators discover it on arrival.

Pre-build gates:

- `CM-README-DOC-LINK-SECTION-PRESENT` — README.md contains the literal `Tracked-Items` heading AND both section markers (`<!-- doc-link-section:begin -->` , `<!-- doc-link-section:end -->`).

- `CM-README-DOC-LINK-ROWS-COMPLETE` — every canonical doc (Issues, Issues_Summary, Fixed, Fixed_Summary, CONTINUATION, every Status.md + Status_Summary.md found under `docs/`) appears as a row inside the section.
- `CM-README-DOC-LINK-FRESHNESS` — the `Last modified` value in each row matches the corresponding doc's §11.4.44 header within sync-wrapper granularity (no row staler than the source doc by more than the sync wrapper's tolerance).
- `CM-COVENANT-114-57-PROPAGATION` — anchor literal across files.

Paired mutations (per §1.1):

- Strip the section markers from README.md → `CM-README-DOC-LINK-SECTION-PRESENT` FAILs.
- Remove the CONTINUATION.md row from the section → `CM-README-DOC-LINK-ROWS-COMPLETE` FAILs.
- Backdate a `Last modified` value in a row → `CM-README-DOC-LINK-FRESHNESS` FAILs.
- Strip `§11.4.57` literal from `constitution/CLAUDE.md` → `CM-COVENANT-114-57-PROPAGATION` FAILs.

No escape hatch. No `--skip-readme-doc-links`, `--collapse-status-rows`, `--no-freshness-check` flag. The discipline exists because invisible docs are non-existent docs from a discoverability standpoint, and the §11.4 covenant specifically prohibits surfaces where claims (here: doc freshness) can drift unnoticed.

Classification: universal (per §11.4.17). Applies to every project that maintains an Issues / Fixed split or Status.md docs (i.e., to every project consuming this Constitution that has any of §11.4.15 / §11.4.45 / §11.4.53 in scope).

2026 05 19

---

§ . . — README always-sync mandate (User mandate, - -)

Forensic anchor — direct user mandate (verbatim, 2026-05-19):

> "fully review and update our main README document. Some points are not valid anymore, some are missing. Make sure main README is among documents we MUST ALWAYS keep updated and in Sync with the projects and other documentation! Make sure we always export it (on every update) into PDF and HTML. This mandatory rules/constraints MUST BE all added into the root (constitution Submodule) Constitution, AGENT.MD and CLAUDE.MD!"

`README.md` at the project root is a §11.4.12-class always-sync document. It MUST be:

1. **Reviewed and updated whenever** new docs / integrations / Status.md entries appear, new submodules land, applied-fixes count changes, or canonical paths shift.
2. **Kept in lockstep** with `docs/CONTINUATION.md` (§12.10) and the Issues / Issues_Summary / Fixed / Fixed_Summary doc set (§11.4.12, §11.4.53) — same always-sync discipline.
3. **Exported to `.html` and `.pdf` on every update**. New helper `scripts/testing/sync_readme_export.sh` performs the pandoc + weasyprint export; it is auto-invoked by `scripts/testing/sync_issues_docs.sh` so a single doc-sync run refreshes the entire doc surface (Issues / Issues_Summary / Fixed / Fixed_Summary / CONTINUATION / README — md + html + pdf).
4. **Carry a §11.4.44 revision header** (Revision N + Last modified ISO timestamp) at the top of the file.
5. **Contain a Documentation Map section** linking to every Status.md
    - Status_Summary.md + spec + plan + guide + script-companion doc + changelog + the constitution submodule, plus a per-audience navigation (end user / developer / QA / agent).
6. **Self-contained** — no hyperlinks to ephemeral external systems as the only source of truth.

Stale exports are §11.4.59 violations regardless of whether the underlying `README.md` is correct — an operator (or future agent) reading the HTML or PDF gets a divergent view, and the §12.10 CONTINUATION resumption guarantee silently breaks. Same discipline as §11.4.12 Issues_Summary and §11.4.53 Fixed_Summary applied to README.md.

Captured-evidence enforcement. Pre-build gate `CM-README-EXPORT-SYNC` locks four invariants: (a) `README.md` exists, (b) `README.html` exists, (c) `README.html` mtime ≥ `README.md` mtime, (d) `README.pdf` mtime ≥ `README.md` mtime (skipped gracefully if weasyprint is unavailable on the host but enforced when the PDF is present). Paired meta-test mutation backdates `README.html` + `README.pdf` so the source is newer → gate FAILs.

Composition. Composes with §11.4.12 (Issues_Summary parity), §11.4.18 (script-companion docs), §11.4.23 (visual-cue HTML colorizer not applicable to README's narrative format), §11.4.44 (revision header on every Status.md and now README.md), §11.4.45 (Status.md integration), §11.4.53 (Fixed_Summary parity — README is the canonical sibling at the project-overview layer), §11.4.56 (Status_Summary parity), §11.4.57 (README.md is the canonical index per Phase 39.EW+1), §12.10 (CONTINUATION.md resumption guarantee depends on README's Documentation Map being current).

No escape hatch. §11.4.59 has NO operator-facing override flag. No `--skip-readme-sync`, `--no-readme-export`, `--readme-stale-OK` flag exists. The discipline exists because README.md is the first file any new agent / operator / contributor reads — letting it diverge from reality is the exact §11.4 PASS-bluff pattern but at the project-overview layer.

Classification: universal (per §11.4.17). Applies to every project consuming this Constitution. The doc-sync helper, pandoc + weasyprint dependencies, and gate name are universal; the specific Documentation Map contents are consumer-side per-project.

Canonical authority: constitution submodule `Constitution.md` §11.4.59.

Non-compliance is a release blocker regardless of context.

§ . . — Documentation always-sync composite covenant (User mandate, - -)

Forensic anchor — verbatim user mandate (2026-05-19 ~09:00Z):

> "Double check if all documents are properly tied with our root Constitution, CLAUDE.MD and AGENTS.MD so they are always up to date, always in sync and exported into PDF and HTML! ... Issues, Issues_Summary, Fixed, Fixed_Summary, Continuation, Status and Status_Summary for all contexts (areas) — THEY ALL MUST BE REGULARLY UPDATED, IN SYNC AND CONSISTENT without giving at any moment false picture about the state of the project or particular area(s) of it!"

The covenant. Eight documentation classes constitute the project's living state surface. They MUST be in sync at all times across markdown + HTML + PDF artefacts. Per-class anchors §11.4.12 / §11.4.44 / §11.4.45 / §11.4.53 / §11.4.56 / §11.4.57 / §11.4.59 / §12.10 each govern one class; §11.4.60 binds them via a single mechanically-enforced composite invariant so the failure mode "one per-class gate silently disabled while operator reads a divergent HTML/PDF" is structurally impossible.

Eight bound doc classes (artefact triple — `.md` + `.html` + `.pdf`):

1. `docs/Issues.md` — open-item tracker (§11.4.12 + §11.4.15 + §11.4.16)
2. `docs/Issues_Summary.md` — auto-generated short form (§11.4.12)
3. `docs/Fixed.md` — closed-item archive (§11.4.53)
4. `docs/Fixed_Summary.md` — auto-generated short form (§11.4.53)
5. `docs/CONTINUATION.md` — resumption handoff (§12.10)
6. `README.md` — project overview + doc-link index (§11.4.57 + §11.4.59)
7. `docs/**/Status.md` (every domain-scoped instance) — per-integration status (§11.4.45)
8. `docs/**/Status_Summary.md` (every domain-scoped instance) — auto-generated short form (§11.4.56)

Mandatory composite gate. `CM-DOCS-COMPOSITE-SYNC` (pre-build, device/rockchip/rk3588/tests/pre_build_verification.sh):

- For every doc-class instance with `.md` present, verify `.html` mtime ≥ `.md` mtime AND `.pdf` mtime ≥ `.md` mtime.
- For every Status.md instance, also verify the sibling `Status_Summary.md` mtime ≥ `Status.md` mtime (if the summary exists).
- Walks `docs/` recursively for `Status.md` — the Status fleet is not enumerated statically because new integrations land per phase.

Composite gate FAILs the build if ANY single instance fails. The per-class gates (`CM-ISSUES-SUMMARY-SYNC` , `CM-DOCS-EXPORT-SYNC` , `CM-FIXED-SUMMARY-SYNC` , `CM-CONTINUATION-DOC-INSYNC` , `CM-README-EXPORT-SYNC`) remain in place — §11.4.60 is the belt-and-suspenders layer that catches what they miss.

Paired mutation (per §1.1). `meta_test_false_positive_proof.sh` backdates `docs/Issues.html` to year 2000 (`touch -t 200001010000.00`) and asserts `CM-DOCS-COMPOSITE-SYNC` FAILs.

Revision-header coupling. Per §11.4.44 every one of the 8 doc classes carries the `**Revision:** N` + `**Last modified:** ISO 8601 UTC` header directly under the H1. `CM-DOCS-COMPOSITE-SYNC` does NOT re-check the revision header (already covered by `CM-DOC-REVISION-HEADER-PRESENT`) — composite gate is artefact-mtime only, by design narrow.

Auto-sync wrapper coupling. Per §11.4.12 every edit to an Issues/Fixed-class doc flows through `sync_issues_docs.sh` . Per §11.4.45 every edit to a Status.md flows through `sync_integration_status.sh <Status.md>` . Per §11.4.59 every edit to README.md flows through `sync_readme_export.sh` . §11.4.60 does NOT change the wrapper contract — it only catches the failure mode where an operator (or AI agent) bypassed the wrapper and committed markdown without regenerating exports.

No escape hatch. No `--skip-composite-doc-sync` , `--allow-stale-html` , `--summary-not-applicable` flag exists. The covenant exists for the operator's own protection — the moment a single per-class gate is bypassed, divergence accumulates silently until release-time forensics finds it (the exact §11.4 PASS-bluff pattern the canonical covenant was authored to eliminate).

Composes with §11.4.12 (Issues_Summary), §11.4.15 (Status field), §11.4.16 (Type field), §11.4.19 (atomic Issues→Fixed migration), §11.4.23 (colorizer), §11.4.33 (type-aware closure vocabulary), §11.4.44 (revision header), §11.4.45 (Status.md), §11.4.53 (Fixed_Summary), §11.4.56 (Status_Summary), §11.4.57 (README doc-link), §11.4.59 (README always-sync), §12.10 (CONTINUATION maintenance).

Propagation. Pre-build gate `CM-COVENANT-114-60-PROPAGATION` enforces the §11.4.60 anchor literal in every CLAUDE.md / AGENTS.md across parent + 10 owned submodules + HelixQA dependencies (same shape as existing §11.4.5X propagation gates). The propagation gate is OPTIONAL for Phase 39.FC's initial landing — only the composite gate + its paired mutation are mandatory; propagation gates trail per established phase pattern.

Classification: universal (per §11.4.17). Applies to every project consuming this Constitution that maintains any of the 8 doc classes. The composite gate logic is universal; the specific doc-class instance paths are consumer-side per-project.

Canonical authority: constitution submodule `Constitution.md` §11.4.60.

Non-compliance is a release blocker regardless of context.

2026 05 19

§ . . — Mandatory Markdown metadata table + structured-doc ToC (User mandate, - -)

Forensic anchor — direct user mandate (verbatim, 2026-05-19):

> "For every Markdown document which contains structured content (with headings / sections and sub-sections) make sure that every time we apply change to the structure, table of contents on the top of the document is created or updated! This is MANDATORY for every structured MARKDOWN document. Automatically its PDF and HTML versions MUST BE (re)generated!"
>
> "Introduce ... revision number, date and time of creation, date and time of last modification, other useful information we have in documents such as Issues, Issues_Summary, Fixed, Fixed_Summary, Status, Status_Summary, Continuation and similar. ... make this mandatory for EVERY Markdown document from now on, update root constitution Submodule with these changes and commit and push it all to all upstreams."

Scope of §11.4.61. This clause adds two universal disciplines that sit alongside (NOT replacing) the §11.4.65 universal export mandate. §11.4.65 governs the existence and freshness of `.html` + `.pdf` siblings; §11.4.61 governs the *content* of the Markdown sources: how revision/status metadata is displayed at the top, and how the document's navigation map (Table of contents) is kept in lockstep with the heading structure. The two clauses compose — §11.4.65 guarantees readers see *something current*; §11.4.61 guarantees what they see communicates its own provenance and is internally navigable.

A. Canonical metadata table (supersedes §11.4.44 format). The §11.4.44 bold-line revision header is superseded by a **Markdown table** placed immediately after the document's H1 title (and any blank line that follows). Pre-existing bold-line headers remain historically valid but MUST migrate to the canonical table at the next substantive edit. The canonical table has these MANDATORY rows:

Field	Value
Revision	positive integer; starts at 1, monotonic, never reset
Created	ISO 8601 date — first commit touching the file
Last modified	ISO 8601 date — most recent commit touching the file
Status	one of `active` / `draft` / `deprecated` / `superseded`

ENCOURAGED rows (REQUIRED when the document tracks workable items or has a known continuation; render with `—` / `none` when N/A):

Field	Value
Status summary	one-line current-state hook
Issues	comma-separated workable-item IDs
Issues summary	one-line summary of open issues
Fixed	comma-separated workable-item IDs of resolved items
Fixed summary	one-line summary of fixes
Continuation	next action or linked CONTINUATION.md anchor

Additional rows MAY be added for project-specific metadata as long as the MANDATORY rows are present.

B. Structured-document Table of contents. Any tracked `*.md` with **two or more H2 sections** ("structured content") MUST include a `## Table of contents` section immediately after the metadata table. The ToC MUST list every H2 and H3 in document order with anchor links and MUST be regenerated on every structural change (heading added / removed / renamed / reordered). A stale ToC is a §11.4 PASS-bluff: operators reading the rendered Markdown see a navigation map that no longer matches the body.

Scope (IN-scope under §11.4.61). Every tracked `*.md` in the §11.4.65 INCLUDED set (project-root `*.md`, `docs/**/*.md`, `scripts/**/*.md` companion docs, owned-submodule trees, `constitution/**/*.md`, owned

HelixQA dependencies). The §11.4.44 exclusions for `CLAUDE.md` / `AGENTS.md` / `README.md` are SUPERSEDED here too — explicit metadata in the file itself is preferred over indirect tracking via a separate VERSION file because operators reading the rendered document see the freshness directly.

Narrow exemptions (NOT IN-scope). Same EXCLUDED set as §11.4.65 (`external/**` , `prebuilts/**` , `out/**` , `build/**` , application- internal `*.md` shipped with source code, non-owned third-party submodule trees). Additional exemptions for the structural rules: `LICENSE` , `LICENSE.md` , `NOTICE` , `VERSION` , `OWNERS` , machine- generated `CHANGELOG.md` .

PDF + HTML siblings. Deferred to §11.4.65. The metadata table and ToC are *content* requirements; their export-freshness guarantee lives in §11.4.65's `CM-UNIVERSAL-MARKDOWN-EXPORT-SYNC` gate. A §11.4.61 metadata or ToC change is a `.md` modification, which transitively triggers §11.4.65 regeneration — the two clauses compose without duplicate enforcement.

Anti-bluff captured-evidence gates (planned):

- `CM-MD-METADATA-PRESENT` — walks every tracked `*.md` (minus exemptions) and asserts (a) H1 present, (b) within 25 lines below H1 a Markdown table contains all four MANDATORY rows (`| Revision |` , `| Created |` , `| Last modified |` , `| Status |`).
- `CM-MD-TOC-PARITY` — for every `*.md` with `≥ 2` H2 sections, asserts a `## Table of contents` is present and its entries' anchor slugs match the document's live H2/H3 set in order.

Paired §1.1 mutation tests (planned):

- Strip the `| Revision |` row from one `*.md` → `CM-MD-METADATA-PRESENT` FAILs.
- Rename one H2 without updating ToC → `CM-MD-TOC-PARITY` FAILs.

Composition with §11.4.44. §11.4.44 remains the authoritative clause for the *fact* that documents must carry revision metadata; §11.4.61 changes the *format* (bold-lines → table) and *scope* (`docs/**/*.md` → every tracked `*.md` per §11.4.65 INCLUDED set). Consumer projects with existing gates that grep for `**Revision:**` MUST update those gates to also accept the canonical table form. Migration period: 30 days from §11.4.61 landing.

Composition with §11.4.45 / §11.4.59 / §11.4.60 / §11.4.65. The per-class always-sync clauses (Status.md §11.4.45, README §11.4.59, composite §11.4.60) and the universal export covenant §11.4.65 govern *artefact existence and freshness*; §11.4.61 governs *content discipline inside the source* `.md`. The clauses are orthogonal layers of the same anti-bluff posture.

No escape hatch. No `--skip-md-metadata`, `--no-metadata-table`, `--toc-stale-OK`, `--allow-missing-revision` flag exists. Divergent metadata and stale ToCs are §11.4 PASS-bluff patterns at the doc-surface layer.

Classification: universal (per §11.4.17). Applies to every project consuming this Constitution. The canonical table format and ToC mandate are universal; the per-project tracker IDs (e.g. `HRD-`) and specific Issues/Fixed-class doc paths are consumer-side per-project.

Canonical authority: constitution submodule `Constitution.md` §11.4.61.

Non-compliance is a release blocker regardless of context.

2026 05 19

§ . . — Workable-items procedure docs as single source of truth (User mandate, - -)

Forensic anchor — verbatim user mandate (2026-05-19 ~10:30Z):

> "To make workable items tracking and creation through all of the mentioned mandatory documents we MUST create proper procedure document which will be named in Constitution, AGENTS.md and CLAUDE.md as single source of truth for this procedure! Make sure it is created under: docs/procedures/issues/Creation.md which will be always exported to PDF and HTML whenever it is updated. We MUST create procedure documents for Updating.md, Resolution.md, Reopening.md and other workable items actions that we have. ... Everything done on project - new features, changes, tasks, bug fixes, investigation, ordinary tasks (writing documentation for example) MUST ALL go through the workable items flow and proper procedures!"

Every workable-item action — opening, updating, closing, reopening, migrating across Issues.md ↔ Fixed.md — MUST follow the canonical procedure document at `docs/procedures/issues/<Action>.md`. The five procedure docs are MANDATORY references; no agent or operator may invent ad-hoc procedures. The closed-set:

Procedure doc	Covers
`Creation.md`	Opening a new workable item (Bug / Feature / Task)
`Updating.md`	Non-closure, non-reopen edits — status transitions, evidence append, type re-classification
`Resolution.md`	Atomic Issues.md → Fixed.md close (Bug→Fixed, Feature→Implemented, Task→Completed per §11.4.33)
`Reopening.md`	Atomic Fixed.md → Issues.md reopen with §11.4.34 source attribution + §11.4.55 reopens-history
`Migration.md`	Bidirectional atomic-move mechanics + sync_issues_docs.sh internals (consumed by Resolution + Reopening)

Each procedure doc carries the §11.4.44 revision header + HTML + PDF exports synchronised to its `.md` source. New helper `scripts/testing/sync_procedure_docs_export.sh` walks the procedure- docs directory and emits matching `.html` + `.pdf` for each `.md`; invoked as the final stage of `scripts/testing/sync_issues_docs.sh` so a single operator invocation refreshes every doc-side artifact.

Universality of scope. All work — new features, behavioural changes, tasks (refactor / doc / infra / gate / audit / cleanup), bug fixes, investigation, documentation work — MUST flow through these procedures. The "I'll just tweak this one file" escape hatch is forbidden because it produces silent doc drift and reopens the §11.4 covenant's PASS-bluff failure mode at the process-tracking layer.

Composes with §11.4.6 (no-guessing — every procedure step is mechanical, no `likely` / `seems`), §11.4.7 (demotion-evidence — Reopening.md is the canonical implementation), §11.4.11 (file-layout — procedure docs live in `docs/procedures/issues/`, qa-results forensics live in `qa-results/`), §11.4.12 (Issues_Summary

sync — all five procedures invoke `sync_issues_docs.sh` whose pipeline is documented in Migration.md), §11.4.15 (Status closed-set — all five procedures cite the same closed-set), §11.4.16 (Type closed-set), §11.4.19 (atomic Issues↔Fixed move — Migration.md is the implementation), §11.4.23 (visual cue + grouping colorization — invoked by sync wrapper, no procedure-side bypass), §11.4.33 (type- aware closure vocabulary — Resolution.md enforces), §11.4.34 (Reopened-source attribution — Reopening.md enforces), §11.4.44 (revision header on every procedure doc), §11.4.53 (Fixed_Summary parity — Migration.md documents the pipeline), §11.4.54 (ATM-NNN identifier — Creation.md allocates), §11.4.55 (Reopens-history per- item doc — Reopening.md authors), §11.4.60 (documentation always- sync composite — procedure docs are an additional doc class binding to the composite covenant).

Pre-build gate `CM-PROCEDURES-DOCS-PRESENT` checks (1) all 5 procedure docs exist at `docs/procedures/issues/{Creation,Updating,Resolution,Reopening,Migration}.md`, (2) each carries the §11.4.44 revision header, (3) each has matching `.html` + `.pdf` exports with mtime ≥ the .md mtime, (4) `scripts/testing/sync_procedure_docs_ export.sh` exists + is executable, (5) `scripts/testing/sync_issues_ docs.sh` invokes `sync_procedure_docs_export.sh`. Paired mutation: rename one procedure doc → gate FAILs.

Propagation gate `CM-COVENANT-114-63-PROPAGATION` enforces this anchor literal in every CLAUDE.md / AGENTS.md across parent + 10 owned submodules + nested submodules + HelixQA dependencies. Paired mutation strips the anchor literal → gate FAILs.

No escape hatch — no `--ad-hoc-procedure`, `--skip-procedure-doc`, `--procedure-not-applicable` flag exists. Inventing an alternate procedure for a "small" change is the exact failure mode this anchor closes.

Classification: universal (per §11.4.17). The closed-set of five procedure docs and the sync-wrapper composition are universal; the specific test-name / fix-name examples within each procedure doc may be consumer-side per-project.

Canonical authority: constitution submodule `Constitution.md` §11.4.63.

Non-compliance is a release blocker regardless of context.

§ . . — Universal Markdown export mandate (User mandate, - -)

Forensic anchor — direct user mandate (verbatim, 2026-05-19):

> "Any markdown document inside the project and which is not part of the applications or services source code MUST BE exported (be available) in PDF and HTML! Any already existing Markdown document that fulfills this condition and which does not have HTML or PDF at all or it is not in sync with it MUST HAVE (re)generated PDF and HTML version! Every time when Markdown document (file) is modified, its proper HTML and PDF versions MUST BE regenerated. Markdown documents MUST BE at all times in sync with PDF and HTML versions!"

The covenant generalizes the per-class always-sync discipline of §11.4.12 / §11.4.18 / §11.4.44 / §11.4.45 / §11.4.53 / §11.4.56 / §11.4.57 / §11.4.59 / §11.4.60 / §11.4.63 / §11.4.64 to **every Markdown document in the project that is not part of an application or service's source-code tree**. Per-class anchors govern specific files (Issues, Fixed, Status, README, etc.); §11.4.65 is the catch- all: any *other* `.md` file that documents the project — guides, research notes, plans, hardware-ID notes, script-companion docs, changelogs, procedure docs, README files inside owned submodules — MUST also have `.html` + `.pdf` siblings, all three artefacts in sync at all times. Stale exports of *any* documentation surface are the exact "operator reads divergent HTML/PDF while .md is correct" PASS-bluff §11.4 forbids — only the per-class enforcement so far prevented it for specifically-named docs; §11.4.65 closes the long tail.

Scope (closed-set):

INCLUDED:

- Project root `*.md` (README.md, CLAUDE.md, AGENTS.md, CONTRIBUTING.md, etc.)
- `docs/**/*.md` (guides, research, plans, changelogs, procedures, hardware notes)
- `scripts/**/*.md` (script-companion docs in documentation format, NOT shebang scripts)

- Owned-submodule trees at `device/rockchip/atmosphere/<submodule>/`: top-level README.md / CLAUDE.md / AGENTS.md / CHANGELOG.md and any `docs/**/*.md`
- `constitution/**/*.md` (the canonical-root submodule)
- `tools/helixqa/<helixqa-submodule>/` top-level README.md / CLAUDE.md / AGENTS.md and any `docs/**/*.md` (owned HelixQA dependencies)

EXCLUDED (NOT subject to §11.4.65 — these are application/service source code or third-party trees we do not own):

- `external/**` (AOSP-mainline / upstream-mirror source trees)
- `prebuilts/**` (binary prebuilts + their bundled docs)
- `packages/modules/**` (AOSP mainline modules)
- `kernel-5.10/**` (kernel source tree — has its own upstream docs)
- `out/**` (build output)
- `build/**` (AOSP build system)
- Application / service source-code trees (e.g. Java/Kotlin module-internal `*.md` shipped with library code, README files inside source-only third-party gradle module directories)
- Any third-party submodule NOT in the owned-submodule set (presenter, vlc-player, nova-player, mpv-player, gramophone-player, rhythm-player, strep-player, smarttube-player, torrserve, lampa) or HelixQA-owned set (Challenges, Containers, DocProcessor, LLMOrchestrator, LLMProvider, VisionEngine)

Mandatory protections (ALL must hold):

1. **Every INCLUDED `.md` file has `.html` and `.pdf` siblings.** A missing export is a §11.4.65 violation regardless of when the markdown was last touched.
2. **`.html` and `.pdf` mtime ≥ `.md` mtime** (within the same sync-wrapper invocation granularity). Stale exports are violations even if the .md itself is correct.
3. **Every modification triggers regeneration.** Whether via direct sync-wrapper invocation (`sync_all_markdown_exports.sh`), a git pre-commit hook auto-regenerating on staged `.md` changes, or the existing per-class wrappers

(§11.4.12 sync_issues_docs.sh, §11.4.18 script-docs sync, §11.4.59 sync_readme_export.sh, etc.) all of which delegate the universal export path to the same canonical helper.

4. **Pre-build gate enforces parity.** `CM-UNIVERSAL-MARKDOWN-EXPORT- SYNC` walks the INCLUDED scope, verifies every `.md` has `.html`
    - `.pdf` siblings with mtime ≥ `.md` mtime, and FAILs the build if any are missing or stale.
5. **No escape hatch.** No `--skip-md-exports`, `--no-pdf-only`, `--md-export-not-applicable`, `--application-internal-doc` flag exists for files inside the INCLUDED scope. The EXCLUDED scope is the only legitimate path to opt out, and it is closed-set.

Canonical helper. `scripts/testing/sync_all_markdown_exports.sh` walks the INCLUDED scope, invokes pandoc (HTML) + weasyprint (PDF) with `timeout 60` each (graceful per-file degradation), uses `docs/_progress-style.css` for visual consistency, and supports `--check-only` (exit nonzero if any out-of-sync, print list) and `--regenerate-all` (force) modes. Caps at 500 candidates with explicit abort+list if the scope is unexpectedly large (e.g. after a new submodule lands without scope-update review). Idempotent.

Composition. Composes with §11.4.12 (Issues_Summary sync — universal helper is the back-end), §11.4.18 (script-companion docs), §11.4.23 (HTML colorizer continues to post-process the always-sync docs for visual cues), §11.4.44 (revision header on every .md the universal helper exports), §11.4.45 (Status.md auto-sync), §11.4.53 (Fixed_Summary), §11.4.59 (README always-sync — the universal helper covers the long tail of every *other* root-level .md too), §11.4.60 (composite always-sync covenant — §11.4.65 is the structural generalization), §11.4.63 (procedure docs — already always-sync; §11.4.65 adds catch-all for every doc class not yet anchored), §11.4.64 (topic discoverability — summary docs the universal helper exports continue to be post-processed by inject_topic_index.py).

Pre-build gates:

- `CM-UNIVERSAL-MARKDOWN-EXPORT-SYNC` — invariants: (a) `scripts/testing/sync_all_markdown_exports.sh` exists + executable, (b) running it in `--check-only` mode returns 0 (no out-of-sync file in the INCLUDED scope).

- `CM-COVENANT-114-65-PROPAGATION` — anchor literal `11.4.65` in every CLAUDE.md / AGENTS.md across canonical-root + parent + 10 owned submodules + nested submodules + HelixQA dependencies (42-file fleet — same propagation set as §11.4.58).

Paired meta-test mutations: one strips the `CM-UNIVERSAL-MARKDOWN- EXPORT-SYNC` gate's enforcement literal; one strips the §11.4.65 anchor literal from a sentinel propagation file. Both mutations assert the corresponding gate FAILs when applied.

Classification: universal (per §11.4.17). The mandate to export every project-doc Markdown to HTML+PDF is universal across every project that consumes this Constitution; the specific INCLUDED / EXCLUDED scope path lists are consumer-side per-project (since each consumer has different submodule topology, build trees, and source- code roots).

Canonical authority: constitution submodule `Constitution.md` §11.4.65.

Non-compliance is a release blocker regardless of context.

2026 05 20

§ . . — Main-specification document versioning + revision discipline (User mandate, - -)

Forensic anchor — direct user mandate (verbatim, 2026-05-20):

> "Make sure everything we add now in previous and upcoming requests IS ALWAYS applied to the main specification — if we have one. Since all these are not major changes we could increase Specification version per change for secondary version instead of the primary. Primary version MUST BE increased for much bigger levels of changes! Add this into root (constitution Submodule) Constitution.md, CLAUDE.md and AGENTS.md as mandatory rule / constraint applicable ONLY IF we have something like the main specification document or we do recognize something like the main specification document. Document MUST BE updated ALWAYS to follow the versioning rules we are appling here + revision and other properties we have!"

Scope condition. This clause applies **only when a project recognises a main specification document** — i.e. a project-level Markdown file (or set of files) that constitutes the canonical "what this system does" reference, typically at `docs/specs/**/specification*.md` or a comparable path. Projects with no main specification (small libraries, internal tooling) are exempt. Projects that operate per-feature spec files (e.g. ADRs only) are exempt unless they elect to adopt the main-spec pattern.

The mandate. When a project DOES have a main specification document:

1. **Every additive operator requirement, refinement, or accepted recommendation MUST be applied to the spec** before, or as part of, the work that implements it. The spec is the source of truth; downstream code and tracking docs reference it.
2. **Spec versioning has two axes** — *primary* and *secondary* (where "secondary" maps to the existing §11.4.61 metadata-table `Revision` row):
    - **Primary (V1 / V2 / V3 / …)** bumps for major rewrites — substantial architecture changes, foundational scope shifts, deprecating large sections, replacing the technology stack. Triggered explicitly by operator decision; old primary versions move to `archive/` per §11.4.61.

    - **Secondary (`Revision`)** bumps for every other change — section additions, mandate landings, structural reorganisation, polish, type-contract refinements. The metadata table's `Revision` integer is the secondary version (matches §11.4.44 monotonic-integer rule).

3. **The metadata table on the spec MUST stay current** — `Revision` bumps in lockstep with the change; `Last modified` updates to the change date; `Status summary` describes the bump's content; `Fixed` and `Fixed summary` reference the relevant work IDs.
4. **Cross-document propagation** — when a project has propagated copies of the spec-change rule (Herald uses CLAUDE.md, AGENTS.md, HERALD_CONSTITUTION.md per its §106), those copies MUST also reference the active spec file (`specification.V<primary>.md`) and not a stale archived version.
5. **Archive semantics** — when the primary version bumps, the old `specification.V<n>.md` moves to `<spec-dir>/archive/specification.V<n>.md` and gains `Status: superseded` with a `Continuation` pointer to the new active file. Per the §11.4.65 universal-export mandate the archived `.html` and `.pdf` siblings travel with it.

Anti-bluff captured-evidence gate (planned). `CM-SPEC-VERSION-DISCIPLINE` :

- If the project advertises a main spec (operator marks via `[project].main_spec_path = "docs/specs/.../specification.V<n>.md"` in a canonical config file), the gate asserts:
    - The file at `main_spec_path` exists.
    - Its metadata table's `Revision` row is ≥ the highest `Revision` referenced anywhere else in the repo (Issues.md, Fixed.md, CLAUDE.md, AGENTS.md).
    - If commits exist that touch the spec since the last `Revision` bump, the gate FAILs (operator forgot to bump).

Paired §1.1 mutation. Backdate the `Last modified` row to a prior date while leaving real-content edits in place → gate FAILs.

Composition. Composes with §11.4.44 (revision header — `Revision` is the secondary version here), §11.4.61 (metadata table + ToC), §11.4.59 (README always-sync — if README cites the spec, the citation MUST point at the current primary version), §11.4.65 (universal export — archived versions stay exported).

Classification: universal (per §11.4.17), applicable conditionally per the scope condition above.

Canonical authority: constitution submodule `Constitution.md` §11.4.73.

1 1 . 4 . 8 6 Non-compliance (forgetting to bump, or letting the spec drift behind operator-mandated requirements) is a release blocker.

2 0 2 6 - 0 5 - 2 5

§ 1 1 . 4 . 8 7 — Roster/corpus-backed Status-doc auto-sync mandate (User mandate, 2 0 2 6 - 0 5 - 2 5)

Forensic anchor — verbatim user mandate (2026-05-25):

> "Make sure that assets and players Status docs are ALWAYS regularly updated and in sync like all others Status docs — any time we add or modify the assets content(s) or we change or add new / remove existing pre-installed video and audio player apps! This MUST WORK OUT OF THE BOX!"

Some Status docs (§11.4.45) are backed not by hand-written narrative alone but by a **tracked roster** (a set of installed apps / components) or a **tracked asset corpus** (a directory of test / media assets). For these, freshness cannot be left to operator vigilance — the moment a member of the roster/corpus changes (a player app added / removed / renamed; a test asset added / modified / removed) the Status doc, its Status_Summary, and their HTML + PDF exports MUST be brought back into sync **out of the box**, mechanically, not by remembering to.

The mandate. Every Status doc whose subject is a tracked roster or asset corpus MUST be kept in sync with that subject by a mechanism where **all** of the following hold:

1. **Drift-proof signal.** A content *fingerprint* — sha256 of the sorted member list, NOT mtime (which `git checkout` resets) — is persisted in a sidecar next to the Status doc. Adding / removing / modifying a member changes the fingerprint deterministically.
2. **Sync helper.** A helper regenerates the fingerprint and re-exports HTML + PDF (composing with the §11.4.65 universal exporter). Invocable standalone AND wired so the sync happens automatically (commit path and/or pre-build).
3. **Enforcement gate.** A pre-build gate recomputes the live fingerprint and FAILs when it differs from the persisted one — forcing the Status doc to be updated whenever the roster/corpus changes. Mirrors §11.4.12 `CM-ISSUES-SUMMARY-SYNC` + §11.4.45 `sync_integration_status`.
4. **Paired §1.1 mutation.** A meta-test mutation corrupts the fingerprint (introducing drift without a doc update) and asserts the gate FAILs — proving the gate is not itself a bluff.

Composes with §11.4.12 (auto-generated docs sync — the canonical sibling pattern), §11.4.45 (integration-status-doc — §11.4.86 is its roster/corpus specialisation), §11.4.53 + §11.4.56 (Summary parity), §11.4.57 + §11.4.59 (README doc-link freshness), §11.4.60 (composite doc sync), §11.4.65 (universal Markdown export), §11.4.6 (no-guessing — the fingerprint is FACT, not a guess about freshness).

Classification: universal (per §11.4.17) — the principle (roster/corpus-backed Status docs auto-sync on subject change) is project-agnostic; the consuming project supplies the specific docs, roster/corpus sources, helper, and gate name in its own CLAUDE.md / AGENTS.md / QWEN.md per §11.4.35.

Canonical authority: constitution submodule `Constitution.md` §11.4.86.

Non-compliance is a release blocker regardless of context. No escape hatch — no `--skip-roster-sync`, `--allow-status-drift`, `--roster-sync-not-applicable` flag exists.

§ . . 2026 05 28 — Latest-Source Documentation Cross-Reference Mandate — instructions, guides, and manuals MUST be verified against the latest official online sources BEFORE publication (User mandate, - -)

Forensic anchor — verbatim user mandate (2026-05-28):

> "Make sure we ALWAYS check against latest versions of services we use web / online docs before creating instructions! This situation is illustration of how we can misguide ourselves or get banned! Add this all important generic / general points as proper mandatory rules when we are creating documentation or guides! These are mandatory rules / constraints and the result is consistency and safety of created instructions, guides and manuals!"

Case study that anchored this rule (Herald 2026-05-28). The first-draft Herald MTProto setup guide (committed at `35fc10c` / `fb00354` / `f089dd6`) recommended VoIP / Google Voice / Twilio / TextNow numbers as a budget-friendly fallback option AND omitted the critical `recover@telegram.org` pre-login email step. Both pieces of guidance directly contradicted (a) Telegram's official documentation at https://core.telegram.org/api/obtaining_api_id ("all accounts that sign up or log in using unofficial Telegram clients are automatically put under observation") AND (b) the gotd/td maintainer's "How to not get banned?" guide (vendored at `submodules/gotd-td/.github/SUPPORT.md`). An operator who followed the original guide could have had their Telegram account permanently banned with no appeal — the very harm the documentation was supposed to help them avoid. The corrected guide landed at `8470ba7` after a forced cross-reference against the latest official sources. This case study is forensic evidence that misguidance-by-stale-docs is the same severity class as a §11.4 PASS-bluff at the documentation layer.

Composition. This anchor composes with §11.4.4 (test-interrupt-on-discovery), §11.4.5 (captured-evidence), §11.4.8 (deep-research validation), §11.4.92 (multi-pass change-evaluation Pass 4), §107 (anti-bluff), and §11.4.98 (full-automation). It closes a specific gap §11.4.92 Pass 4 alludes to but does not explicitly mandate: **every operator-facing instruction, guide, manual, troubleshooting cookbook,**

or setup walkthrough MUST be cross-referenced against the LATEST official online documentation of the service / library being documented BEFORE the document is committed.

(A) Binding rule — pre-commit cross-reference. Before committing any document that contains operator-actionable steps (setup walkthroughs, integration guides, API how-tos, credential acquisition, security configurations, troubleshooting cookbooks, billing-policy documentation, terms-of-service summaries, any external-service interaction guide), the author MUST:

1. **Fetch the latest official online documentation** of the third-party service / library being documented via WebFetch, direct browsing, the service's own MCP server (if available), or equivalent authoritative real-time source. Do NOT rely on training data, memory, prior assumptions, or older committed docs as the source of truth.
2. **Cross-reference each instruction in the new document** against that source. For each step the operator will take, verify: (a) the service still supports that action; (b) the form fields / parameters / endpoints are still the same; (c) any new constraints, deprecations, or warnings the service published since the doc's last verification.
3. **Seek secondary authoritative sources when the official documentation is sparse / silent** on a critical requirement. Examples: the library maintainer's official guides (`submodules/<lib>/README.md` , `SUPPORT.md` , `SECURITY.md`), the service's official blog / changelog, the service's official support email / channel responses, community-vetted FAQs (Stack Overflow accepted answers, official Discord pinned messages).
4. **Cite the source URL + the date checked at the bottom of the document** in a `## Sources verified` section. Example: `Sources verified 2026-05-28: https://core.telegram.org/api/obtaining_api_id + submodules/gotd-td/.github/SUPPORT.md` . The citation is the audit trail; a doc without it is by §11.4.99 definition NOT verified.
5. **Cite the cross-reference in the commit message footer.** A commit that adds or modifies operator-facing instructions MUST include a `Sources verified <date>: <urls>` footer line so the verification trail is reachable from `git log` queries.

(B) Negative-finding documentation is required. If the cross-reference reveals the official source is silent / contradictory / outdated on a question the new document needs to answer, the document MUST explicitly note that gap so the next reader does not assume the absence of contradiction means authoritative agreement. Example phrasing the Herald MTProto guide uses: *"The official Telegram docs do not enumerate App title validation rules; the constraints below were verified against the gotd/td community channel + empirical operator testing 2026-05-28."*

(C) Re-verification cadence. Documents older than 6 months without re-verification are STALE — operators MUST NOT trust them for fresh action without re-running the cross-reference. Documents MUST be re-verified:

1. **Before being cited as the authority for an operator-action campaign** (the operator's command to "follow this guide" is the trigger).
2. **At every major release boundary** (vN.0.0 tag — the release-gate sweep includes a §11.4.99 freshness audit).
3. **Whenever the documented service publishes a breaking-change announcement** (the agent that knows about the announcement MUST queue the re-verification).
4. **When an operator reports an error following the guide** (the case-study anchoring is automatic and triggers immediate re-verification).

(D) Service-specific risk-classifications. Documentation for the following service families MUST include explicit safety warnings cross-referenced against the latest published policies, with a §11.4.99 "Sources verified" date NEVER older than 90 days:

Family	Specific risks the documentation MUST address
Telegram / WhatsApp / messenger APIs (unofficial clients)	Anti-abuse-system observation; one-phone-one-app-id limits; ban-on-VoIP policies; rate-limit floods; user-impersonation risks; pre-login declaration emails.
Cloud provider APIs (AWS, GCP, Azure, Cloudflare)	Billing blast-radius; account-lock policies; service-quota limits; least-privilege IAM; data-residency obligations.
Payment systems (Stripe, PayPal, banks, Mercado Pago)	KYC/AML compliance; PCI-DSS data-handling rules; webhook signature verification; idempotency keys; refund/chargeback flows.
AI / LLM providers (Anthropic, OpenAI, Google AI)	Terms-of-service violation triggers; rate-limit + quota policies; data-retention defaults; safety-classifier outcomes; PII-in-prompt risks.
Code-hosting services (GitHub, GitLab, GitFlic, GitVerse, Bitbucket)	Token-leak revocation policies; force-push to protected-branch policies; rate-limit-on-API-tokens; secret-scanning bot behaviour.
OS / package managers (apt, brew, npm, pip, cargo)	Supply-chain compromise vectors; signature verification; lockfile discipline; mirror-trust policies.

This list is **NOT exhaustive** — when a new external service is documented, the §11.4.99 author judges whether the service has comparable risk surface; if yes, the same safety-warning requirement applies.

(E) Composition with §11.4.92 multi-pass change-evaluation. §11.4.92 Pass 4 (deep-research validation) already requires "external precedent or literal 'NO external solution found'". §11.4.99 strengthens this specifically for the operator-facing-instructions class: deep research is mandatory for ALL such docs, NOT just non-trivial code changes. The agent that authors an instruction guide CANNOT cite §11.4.92 Pass 4 as a substitute for §11.4.99 — the two pass independently.

(F) Inheritance per §11.4.35. Every consuming repository's CLAUDE.md / AGENTS.md / QWEN.md MUST carry a short-form restatement citing the literal anchor `11.4.99`. The pre-build gate `CM-COVENANT-114-99-PROPAGATION` (when implemented) enforces this literal anchor presence across the canonical fleet. Paired §1.1 meta-test mutations strip the load-bearing literal → gates FAIL.

(G) Enforcement. A commit that adds or modifies operator-facing instruction documentation without (a) a "Sources verified " footer in the document itself AND (b) a `Sources verified` line in the commit-message footer is blocked at release-gate. A document with operator-actionable steps that becomes stale (older than 6 months without re-verification, or 90 days for risk-classified services per §(D)) graduates to §11.4.90 Obsolete with `Obsolete-Details: Reason=stale-documentation; superseded-by=<replacement-doc-or-rewrite>` after the 30-day grace window per §11.4.90.

Canonical authority: this Constitution.md §11.4.99 in the HelixConstitution submodule. All consuming projects (Herald, a consuming project, future) restate + cite via §11.4.35 inheritance.

Non-compliance is a release blocker. No `--skip-source-check` / `--documentation-freshness-optional` / `--cite-sources-later` / `--trust-prior-doc-as-authoritative` flag exists. The 2026-05-28 user mandate is unambiguous: the result MUST be "consistency and safety of created instructions, guides and manuals".

11.4.106 2026-05-31

§11.4.106 — Docs Chain — mechanical documentation/DB sync engine (Operator mandate, 2026-05-31)

Short tag: `docs-chain-mechanical-sync`.

Forensic anchor — operator mandate (2026-05-31):

> Docs Chain is the canonical mechanical enforcer of the documentation-sync mandates. Consuming projects MUST use the engine instead of ad-hoc per-project doc-sync scripts; they register their own chains via per-context YAML; the engine never fakes a transform and every `sync` / `verify` carries real captured evidence.

Docs Chain — the `vasic-digital/docs_chain` engine (a universal, Go-implemented, **bidirectional document-and-database dependency-propagation engine**) — is the canonical mechanical enforcer of the family of documentation-sync mandates this Constitution already declares. This is a MANDATORY constraint binding every consuming project. The detailed normative spec is the engine's own documentation set — `docs/CONSTITUTION_INTEGRATION.md` (distribution + inheritance + the anchor mapping table) and `docs/USE_CASE_CATALOGUE.md` (the ready-to-use chain recipes); this section restates its load-bearing constraints, it does not redefine them. Implementations code against the engine's contract, not against a paraphrase.

(A) Use the engine, never ad-hoc scripts. Consuming projects MUST drive their documentation/DB sync through Docs Chain — consumed **by reference** via the flat-layout sibling (the `~/Projects/docs_chain` working tree during local development / the path the constitution exposes once the Phase-6 submodule pointer lands), inherited the same way §11.4.80 inherits the `codegraph_*` scripts: referenced at the constitution-exposed path, **NEVER copied** into the consumer. Ad-hoc per-project doc-sync scripts (`sync_issues_docs.sh` , `generate_*_summary.sh` , `sync_*_export.sh` , `update_readme_doc_links.sh` , et al.) are superseded by the engine and MUST be retired as each matching context is registered.

(B) Consumer-owned contexts via config. The engine is project-agnostic; it carries NO project-specific path or chain. The consumer registers its chains as data via `.docs_chain/contexts/*.yaml` at its own project root — one independent context per file — per the §11.4.28 decoupling rule (the submodule never carries project-specific context; the consumer injects its chains via config). `state.json` (at `<project-root>/.docs_chain/state.json` , regenerated on demand by `docs_chain sync`) and the `*.docs_chain.tmp` staging sidecars are **gitignored**; the consumer owns those `.gitignore` entries.

(C) The sync anchors it mechanizes. Docs Chain is the mechanical engine behind a family of existing documentation-sync anchors; once a consumer registers the matching context it enforces the anchor **in place of the ad-hoc script** — see the `docs/CONSTITUTION_INTEGRATION.md` mapping table: §11.4.12 (Issues_Summary always-sync), §11.4.53 (Fixed_Summary parity), §11.4.45 (Status.md maintenance), §11.4.56 (Status_Summary two-audience parity), §11.4.57 (README doc-link section), §11.4.59 (README always-sync export), §11.4.60 (documentation composite-sync), §11.4.65 (universal markdown export), §11.4.86 (roster/corpus auto-sync), §11.4.93 (workable-items DB single source of truth), §11.4.95 (DB tracked + WAL-checkpoint commit), §12.10 (CONTINUATION maintenance), and §11.4.44 (document revision header — exports kept in sync). Engine-wide guarantees: change detection by **content hash, NOT mtime** (§11.4.86); **atomic-rename + SQLite-transaction commit** with rollback on any transform error and optional hardlink backup (§9.2 zero-risk data safety); a both-dirty `sync` pair surfaces a **conflict — never a silent merge** (exit 2, §11.4.6 no-guessing); `verify` is the deterministic CI / pre-build sink-side gate over byte-stable transforms (§11.4.50); per-run captured evidence lands at `qa-results/docs_chain/<run-id>/` (§11.4.69).

(D) It does NOT replace authoring discipline. Docs Chain replaces only the **mechanical sync** an anchor requires — it does NOT replace the authoring discipline of any anchor. The source author still writes the §11.4.44 revision header into the source; the engine then keeps every derived export in sync. A consumer must not treat the engine as licence to skip authoring.

(E) Anti-bluff (MANDATORY, composes with §11.4 / the end-user quality covenant). Docs Chain NEVER fakes a transform: a missing `pandoc` / `weasyprint` (or any absent tool) surfaces a typed `ToolAbsentError` (`IsToolAbsent`) and an honest §11.4.3 SKIP-with-reason — **never a fake PASS**, never a partial write. Every `sync` / `verify` run carries real captured evidence (the `qa-results/docs_chain/<run-id>/` artefact). A metadata-only / "absence-of-error" / config-only PASS at the sync layer is a §11.4 PASS-bluff.

Status note (§11.4.6 no-guessing). The engine's Phases 1–3 (core DAG + content-hash + adapters + atomicity/orchestrator) AND the `cmd/` CLI / per-context YAML loader (Phase 4) are IMPLEMENTED + tested — `cmd/docs_chain/main.go` registers the `sync` / `verify` / `diff` (alias of `verify`) / `doctor` subcommands wiring the config loader → state →

orchestrator, and `go test -count=1 ./...` is GREEN at 7/7 packages (incl. `internal/orchestrator` spec_wiring_test, both PASS). FACT (§11.4.6): this CLI/loader lives in the working tree but is UNTRACKED (`git ls-files docs_chain/cmd/docs_chain/main.go` is empty) and the engine is NOT yet a registered git submodule (`git submodule status` shows no docs_chain entry; in-tree at `./docs_chain` , HEAD = parent HEAD). The constitution-submodule distribution (Phase 6) remains PLANNED + OPERATOR-GATED. This anchor states the binding contract; consumers wire the engine as each phase lands, and MUST NOT claim working behaviour a phase has not yet shipped.

Classification: universal (§11.4.17) — a content-hash bidirectional doc/DB propagation engine is a vendor-neutral mechanism reusable by ANY governed project. Projects with no derived-export or DB-sync surface inherit the anchor latently (it binds the moment they ship one) — the §11.4.96 "the principle binds even absent the surface" restatement pattern.

4-layer coverage per §11.4.4(b). Propagation gate `CM-COVENANT-114-106-PROPAGATION` enforces the literal anchor `11.4.106` across the canonical consumer fleet (parent + owned-submodule CLAUDE.md / AGENTS.md / QWEN.md). Paired §1.1 meta-test mutation strips the `11.4.106` literal from a consumer file → the gate FAILs. (Gate-code implementation lands as a separate work item; this anchor defines the contract.)

Composes with §11.4 + §11.4.1..§11.4.16 (end-user quality / anti-bluff — sub-rule (E) is bound by it), §11.4.6 (no-guessing — both-dirty `sync` → conflict, never silent merge), §11.4.12 / §11.4.45 / §11.4.53 / §11.4.56 / §11.4.57 / §11.4.59 / §11.4.60 / §11.4.65 / §11.4.86 / §11.4.93 / §11.4.95 / §12.10 / §11.4.44 (the documentation-sync anchors it mechanizes), §11.4.28 (engine/context decoupling), §11.4.80 (inherited-by-reference, never-copied — the codegraph pattern), §9.2 (atomic commit + rollback), §11.4.50 (deterministic `verify` gate), §11.4.69 / §11.4.5 (captured evidence), §1.1 (paired-mutation propagation proof).

(F) Write-seam hook enforcement + real-time-sync honest boundary (extension, 2026-07-15; operator mandate — the Continuation / memory / knowledge artefacts MUST ALWAYS be in sync). The documentation/DB sync this engine mechanizes MUST be ENFORCED AT THE WRITE SEAMS, never left to agent vigilance: the consuming project MUST wire the engine's `verify` (and, where safe, `sync`) into (1) the COMMIT seam — a pre-commit / commit-wrapper hook that REFUSES a commit whose staged set touches a chain SOURCE while

its derived exports / DB rows are stale; (2) the BUILD seam — the pre-build gate; and (3) the CONSTITUTION-PULL seam — the §11.4.164 `post_update_hook.sh` invocation. A stale `CONTINUATION.md` (§12.10), a stale standing session-resumption file (§11.4.131), or a stale memory / knowledge artefact while the live state has ALREADY moved is a violation of the SAME severity class as a stale export — the resumption guarantee silently breaks and a fresh session resumes on fiction. Honest boundary (§11.4.6): hook-wired seam enforcement makes the sync EVENTUALLY-CONSISTENT-AT-EVERY-WRITE — it is NOT literal continuous real-time; genuine continuous real-time sync requires a filesystem-watch daemon (inotify / FSEvents watcher running the chain on change). A project that CLAIMS "real-time sync" while shipping only seam hooks is bluffing the mechanism: state the seam-based guarantee honestly, and carry the watch daemon as a TRACKED upgrade path (§11.4.197 — driven to completion or explicitly closed, never left un-wired).

Canonical authority: this Constitution.md §11.4.106 in the HelixConstitution submodule; detailed spec the Docs Chain engine docs `docs/CONSTITUTION_INTEGRATION.md` + `docs/USE_CASE_CATALOGUE.md` (`vasic-digital/docs_chain`). All consuming projects restate + cite via §11.4.35 inheritance.

Non-compliance is a release blocker. No escape hatch — no `--ad-hoc-sync-ok` , `--skip-docs-chain` , `--fake-transform` , `--sync-evidence-optional` flag exists.

2026 06 15

§ . . — Comprehensive per-feature Status + Status_Summary document set with mandatory video-recording confirmation (User mandate, - -)

Forensic anchor — verbatim user mandate (2026-06-15): "We MUST CREATE proper Status and Status_Summary documents under docs/features containing and covering all system components, all client apps, and all features we have and we have ported from all cli_agents! ... in-depth with no a single feature left ... table ... always up to date and in sync ... status of its usability implementation tests coverage and the confirmation of practical use confirmed on video recording with

no false results or bluff of any kind! ... tightly connected using docs_chain Submodule ... Exitence of such Status (with Status_Summary) document with all exported files (PDF, DOCX, HTML) is mandatory for every single project."

Every project MUST maintain, under `docs/features/`, a comprehensive **feature Status document set** (`Status.md` + its §11.4.56 `Status_Summary.md` companion) that enumerates EVERY system component, EVERY client application/binary/surface (TUI / CLI / Web / desktop / mobile / API / gRPC / library / submodule / infrastructure), and EVERY feature — including features ported in from any incorporated CLI-agent / submodule catalogue (§11.4.74) — with NO single feature left out. The mandate (ALL hold):

(1) **Total, categorized feature coverage** — a per-feature table covering the FULL feature surface, organized Component → Category, reconciled against the actual codebase (a feature present in code but absent from the table, or a table row with no code, is a §11.4.153 violation; "looks complete" without a code-reconciliation pass is a §11.4.6/§11.4.118 bluff).

(2) **Per-feature fields** — each feature row MUST carry: Component, Feature, Category, Implementation status, Wiring status (genuinely reachable by an end user, not merely compiled — §11.4.108), Real-use status, Tests-coverage status (the four-layer §11.4.4(b) coverage state), Validation status (closed vocabulary PASS / FAIL / SKIP / PENDING_FORENSICS / OPERATOR-BLOCKED per §11.4.45), and **Video-recording confirmation** (a path to the captured real-use video proving the feature works for the end user, or an honest gap marker).

(3) **Mandatory per-feature real-use video confirmation** — each user-visible feature's "confirmed working" claim MUST be backed by a recorded real-use video showing a genuine end-user scenario: real prompts → real LLM/service responses producing real results, NEVER a frozen/stale frame (§11.4.107 liveness), NEVER a faked, mocked, demo-loop, or bluff response, NEVER an LLM error/garbage response passed off as success. Videos are captured evidence per §11.4.2/§11.4.5 and stored at the project-declared recording path (§11.4.35); a "confirmed" row with no real video, or a video whose content is a bluff, is a §11.4 PASS-bluff at the feature-status layer. Where autonomous video capture is genuinely infeasible the row is an honest §11.4.3/§11.4.52 SKIP-with-reason + tracked migration item — NEVER a faked confirmation.

(4) **Mandatory video-analysis remediation loop** — every recorded video MUST be analysed (its presented data recognized + checked); any defect the video surfaces MUST trigger the §11.4.102 systematic-debugging → fix → §11.4.146 retest → re-record loop, iterating to a clean GO per §11.4.134, before the feature is marked confirmed. A video that exposes a broken feature is a §11.4.4 test-interrupt, not a confirmation.

(5) **Always-in-sync, mechanically enforced** — the feature Status set is a §11.4.45-class roster/corpus-backed Status doc: kept in sync via the §11.4.106 docs_chain engine + a §11.4.86 drift-proof fingerprint (sha256 of the sorted feature-key roster AND the sorted video-artefact roster, NOT mtime) so that adding/removing/renaming a feature OR a video artefact mechanically re-syncs the docs + exports out-of-the-box; stale = violation.

(6) **Four-format export** — the feature Status set MUST export to HTML + PDF + DOCX (this doc class ADDS DOCX to the §11.4.65 universal HTML+PDF export set; the other doc classes are unchanged), all kept in sync per §11.4.60.

(7) **Follows the Status-doc family** — composes §11.4.44 (revision header), §11.4.45 (Status maintenance + captured-evidence table + operator-blocked-at-top), §11.4.56 (Status_Summary two-audience parity), §11.4.57 (README doc-link row), §11.4.59/§11.4.60 (always-sync composite).

(8) **MP4 format REQUIRED.** All video confirmations MUST be in `.mp4` format (H.264, `movflags +faststart`, `pix_fmt yuv420p`). Window-specific capture ONLY (§11.4.159(A)). Vision validation REQUIRED (§11.4.159(D)). `.cast` (asciinema) files are supplementary only — the primary evidence is the `.mp4` video.

Honest boundary (§11.4.6): the feature Status set guarantees a complete, video-confirmed, always-synced ledger of what works for the end user — it does NOT itself prove correctness of any single feature (that rests on the feature's own §11.4.69/§11.4.107/§11.4.123 captured evidence the row cites), and it does NOT replace §11.4.40 full-suite retest. Classification: universal (§11.4.17) — the consuming project supplies its concrete `docs/features/` layout, feature roster source, recording path, and DOCX exporter per §11.4.35. Composes §11.4.2 / §11.4.5 / §11.4.44 / §11.4.45 / §11.4.52 / §11.4.56 / §11.4.57 / §11.4.59 / §11.4.60 / §11.4.65 / §11.4.86 / §11.4.102 / §11.4.106 / §11.4.107 / §11.4.108 / §11.4.118 / §11.4.123 / §11.4.134 / §11.4.146 / §1.1. Propagation gate `CM-COVENANT-114-153-`

`PROPAGATION` (literal `11.4.153`) + recommended gates `CM-FEATURE-STATUS-COMPLETE` (every code-present feature has a Status row + every row maps to code) + `CM-FEATURE-STATUS-VIDEO-CONFIRMED` (every user-visible confirmed row cites a real captured video or an honest SKIP) + paired §1.1 meta-test mutation (strip the literal → propagation gate FAILs; mark a feature confirmed with no real video / drop a code-present feature from the table → the feature-status gates FAIL; gate-code = separate work item).

Canonical authority: constitution submodule `Constitution.md` §11.4.153. Non-compliance is a release blocker. No escape hatch — no `--skip-feature-status`, `--feature-without-video`, `--frozen-video-OK`, `--bluff-response-video-OK`, `--skip-video-analysis`, `--feature-ledger-incomplete-OK`, `--no-docx-export`, `--allow-feature-ledger-drift` flag.

2026 06 23

§ . . — Exported-document independent content + textual + full-visual validation mandate (User mandate, - -)

Forensic anchor (FACT, 2026-06-23). A "green" Mermaid export fix shipped RAW gantt-diagram source — the literal lines `gantt`, `title`, `dateFormat`, `section`, `:done,` — as plain readable text into user-facing PDFs, unreadable to any reader who expected a rendered diagram. The fix's validation grepped the HTML for `class="mermaid"` but NEVER opened the EXPORTED PDF to check whether raw diagram source had leaked into the body. The operator caught it (§11.4.138 operator-escape — the green suite missed it). Root lesson: generated/exported documents were never INDEPENDENTLY validated for the VISUAL + TEXTUAL correctness of the EXPORTED artifact itself — a `grep` of the source-or-intermediate HTML is NOT a check of what the reader actually receives. This is the §11.4.108 SOURCE→ARTIFACT gap (the change present in source / intermediate, but the rendered bytes a reader opens are broken) materialised at the documentation-export layer.

The mandate. Every generated/exported document — HTML, PDF, DOCX, or any format in the §11.4.65 universal-Markdown-export scope (and the §11.4.153 four-format feature-Status set that adds DOCX) — MUST pass INDEPENDENT validation by a review agent structurally separate from the generator (§11.4.70/§11.4.20 dedicated subagent or a distinct human; NEVER the author/generator

self-checking its own output — §11.4.92 self-evaluation PRECEDES and never satisfies; this is the §11.4.142 every-change-reviewed discipline applied to exported artifacts). The validation MUST be run on BOTH the source Markdown AND every exported artifact derived from it, ALWAYS — every export, not a sampled subset — across THREE layers, ALL of which MUST hold:

(1) CONTENT layer. The export faithfully carries the source's intent + data: nothing dropped, truncated, re-ordered into nonsense, or garbled. Every section, table, list, code block, figure caption, and link the source declares is present and complete in the export (a table truncated to its first page, a list collapsed, a section silently omitted, a link stripped of its target are all CONTENT findings). Cross-check the exported artifact against the source — present-and-complete, not merely "the file exists and is non-empty" (the §11.4.5/§11.4.69 captured-evidence quality bar, not a presence-only bar).

(2) TEXTUAL layer. The export is human-readable: NO raw markup, NO diagram source, NO unrendered code-fence body leaking as ordinary body text. Specifically forbidden as body text in the exported artifact: raw Mermaid diagram source (`mermaid` , `gantt` , `graph` , `flowchart` , `sequenceDiagram` , `classDiagram` , `stateDiagram` , `erDiagram` , `pie` , `gitGraph` , `journey` , `dateFormat` , `section …` , `:done,` / `:active,` / `:crit,` task lines, and the closing-fence residue), raw HTML tags rendered as visible text, raw LaTeX/math source where math rendering was intended, and any triple-backtick code-fence whose intended rendering (diagram / formatted block) was supposed to replace the source. The 2026-06-23 raw-gantt-in-PDF failure is the canonical TEXTUAL finding. Detection reads the exported artifact as text (e.g. `pdftotext` on the PDF, the rendered DOM/text of the HTML, the extracted document text of the DOCX) and asserts the forbidden tokens do NOT appear as body text.

(3) FULL VISUAL layer. Embedded diagrams render as IMAGES, not as source; layout is intact; no overlapping / cut-off / clipped / garbled / collapsed / blank content. Verified by RENDERING the export and inspecting the rendered result — never by inspecting the source or an intermediate alone: e.g. `pdftotext` to catch raw diagram source leaking as text (composes layer 2), `pdfimages` (or the format's image-enumeration) to CONFIRM the intended diagrams are present as rendered raster/vector images (a diagram that should be an image but produces zero embedded images is a VISUAL finding — the source leaked instead of rendering), and `pdftoppm` → OCR (§11.4.117/§11.4.107(12), confidence floor +

ROI) to CONFIRM the rasterised page carries human-readable visual content and to re-catch any raw-source-as-pixels. Captured evidence per §11.4.5/§11.4.69/§11.4.107 (`pdftotext_body.txt` , `pdfimages_manifest.txt` , `ocr_pages/` , the per-finding pinpoint — which page/region/token, expected vs actual) accompanies every verdict.

(4) Anti-bluff — the validator is itself anti-bluff. A rubber-stamp "the export looks fine" is NOT validation (§11.4 / §11.4.1 — a rubber-stamp validator is a PASS-bluff at the export-validation layer). The analyzer/validator is self-validated with a golden-good / golden-bad fixture pair per §11.4.107(10): a golden-good export (clean rendered diagram + faithful content) MUST PASS, and a golden-bad export (a PDF/HTML/DOCX deliberately seeded with raw Mermaid `gantt` source as body text, plus a truncated table) MUST FAIL — a validator that passes its golden-bad fixture is a bluff gate and is investigated per §11.4.102 before any further use. The validation iterates to a clean GO per §11.4.134: any finding in ANY of the three layers (BLOCKING / nit / warning) re-arms the loop — the generator/author fixes it (e.g. pre-renders the diagram to an image before export), the export is regenerated, and the validator RE-VALIDATES, until GO with ZERO findings and ZERO warnings, every verdict backed by captured rendered-artifact evidence. A generated document that ships WITHOUT this independent content+textual+visual validation, or that contains raw diagram source / unreadable garble / dropped content in the exported artifact, is a §11.4 PASS-bluff at the documentation layer.

Honest boundary (§11.4.6). This validation confirms the EXPORTED artifact a reader opens carries the source's content faithfully, is textually readable, and renders its visuals as images — it does NOT prove the source's content is itself correct (that rests on the source's own §11.4.142 review + §11.4.135 regression guards) and does NOT replace the §11.4.65 mtime-parity export-sync gate (which proves the exports are FRESH, not that they are READABLE — §11.4.168 is the READABILITY/FIDELITY layer the mtime gate cannot see). A FLAG_SECURE / DRM-blanked / sink-blanked rendering surface that cannot be rasterised documents the gap per §11.4.112 and uses the §11.4.117 proxy oracle — never a faked visual PASS. "The HTML had `class="mermaid"` " is a source-side check, NOT an exported-artifact check (the exact 2026-06-23 omission); " `pdftotext` shows no `gantt` source AND `pdfimages` shows the expected diagram image AND OCR reads human text" is the exported-artifact check.

Classification: universal (§11.4.17) — a platform-neutral exported-document-fidelity discipline reusable by ANY project that generates/exports documents; the consuming project supplies its concrete pre-render pipeline, exporters, and rendered-artifact validators (text-extraction / image-enumeration / rasterise→OCR) per §11.4.35. Composes §11.4.65 (the export-scope + freshness layer this READABILITY layer sits atop) / §11.4.73 (project-wide MD/HTML/PDF/DOCX styling — styled exports MUST still pass the three layers) / §11.4.107 (the self-validated-analyzer + liveness/OCR validation techniques) / §11.4.117 (CV/OCR pixel-oracle for rasterised-page validation) / §11.4.135 (a fixed export-fidelity defect registers a permanent regression guard) / §11.4.138 (the 2026-06-23 operator-escape that motivated this anchor) / §11.4.142 (independent every-change review applied to exports) / §11.4.159 (window-scoped recording + vision-validation discipline, the recording analogue) / §11.4.163 (the media-validation pipeline — §11.4.168 is its document-export sibling) / §11.4.165 (the independent verification agent — §11.4.168(c) DOCS layer is its export-fidelity refinement) / §1.1.

Propagation gate `CM-COVENANT-114-168-PROPAGATION` (literal `11.4.168` across the consumer fleet) + recommended gate `CM-EXPORTED-DOC-VISUALLY-VALIDATED` (every exported document in the §11.4.65 scope carries an independent-validation marker — produced by a validator structurally separate from the generator — asserting the three layers PASS on the exported artifact, with captured rendered-artifact evidence; the validator is self-validated by a golden-good/golden-bad fixture pair) + paired §1.1 meta-test mutation (strip the literal `11.4.168` → propagation gate FAILs; produce a PDF/HTML/DOCX containing raw diagram source — e.g. raw Mermaid `gantt` lines — as body text and assert the gate catches it → `CM-EXPORTED-DOC-VISUALLY-VALIDATED` FAILs; feed the validator its golden-bad fixture → self-validation FAILs; gate-code = separate work item).

Canonical authority: constitution submodule `Constitution.md` §11.4.168. Non-compliance is a release blocker regardless of context. No escape hatch — no `--skip-visual-validation`, `--raw-source-in-pdf-ok`, `--textual-check-suffices`, `--self-validate-export`, `--diagram-source-as-text-ok` flag exists.

§11.4.186 — Anti-divergence enforcement: cross-document consistency is a mandatory-before-export/commit gate, never an after-the-fact audit (research-derived, 2026-07-08). Compact summary: any project maintaining more than one

representation of the same tracked data (a delivery plan + its MVP brief + a workable-items DB + their summaries + `.md` / `.html` / `.pdf` / `.docx` exports) MUST enforce cross-document CONSISTENCY as a deterministic gate that runs BEFORE any export render, any doc/DB sync `verify`, and any doc-set commit — NEVER as an after-the-fact human audit that runs only when an operator happens to ask (the forensic FACT: the same NanoKVM/phone-as-screen feature appeared in three plan clusters — one shared ticket SPK-596 across SIX rows split over TWO releases, and a carve-out due to ship 2026-08-31 whose own prerequisite does not START until 2026-10-05 — and NO gate caught it; only an operator's manual read of `Plan_v9.0.xlsx` surfaced it, after the divergent bytes had already exported). (1) **One no-divergence verdict, three seams** — a single deterministic PASS/FAIL/SKIP verdict (built by composing a cross-document integrity validator with the workable-items-DB internal `validate`) gates all three write-seams (export / sync-verify / commit); a FAIL at any seam refuses that seam, naming the exact offending records (§11.4.6 actionable). (2) **Five decidable check families** — DEDUP (no two records describe one feature with divergent timeline/status/release, keyed on ticket OR normalised `(subject, scope)`, NOT bare subject substring), TIMELINE (start ≤ deadline; no deadline-before-dependency; no dependency cycle; GATED exempt-but-marked), CROSS-DOC (the same item across every representation carries identical timeline/status/type against a NAMED authoritative source), INTEGRITY (no orphan refs; Status↔Type §11.4.33; location↔status), STRUCTURAL (required columns; ID uniqueness+monotonicity §11.4.54). (3) **Drift-proof trigger** — a §11.4.86 sha256-of-sorted-members fingerprint of the authoritative inputs re-arms the gate on ANY input change, so a stale verify cannot pass on changed data. (4) **Self-validated analyzer (§11.4.107(10))** — the validator ships golden-good (MUST PASS) + one golden-bad per family (MUST FAIL, pinpointing the offender) + a negative-control (distinct same-subject tasks that MUST PASS — the false-positive guard); a validator that passes its golden-bad, or fails golden-good/the negative-control, is itself a §11.4 bluff and a release blocker. (5) **Reusable + decoupled (§11.4.28/.31)** — packaged as a depth-1 reusable engine under the constitution submodule per the §11.4.28(C) carve-out with a `helix-deps.yaml`; project-specific doc-sets + authoritative-source bindings live in a consumer-owned checkset (data, never engine code). (6) **Supersedes ad-hoc audits** — the per-cycle `*_INTEGRITY_FINDINGS.md` / `RECONCILIATION_*.md` after-the-fact audit docs are retired in favour of the gate; honest boundary §11.4.6 — the gate proves internal CONSISTENCY (no record diverges, no timeline is incoherent, every ref resolves), NOT plan CORRECTNESS (achievability/date-

rightness stay operator/management decisions the gate SURFACES, never MAKES). Classification: universal (§11.4.17). Composes §11.4.6/.12/.33/.44/.50/.53/.54/.60/.65/.73/.75/.85/.86/.91/.93/.95/.106/.107/.108/.110/.148/.176. Propagation gate `CM-COVENANT-114-186-PROPAGATION` (literal `11.4.186`) + recommended gate `CM-DOC-INTEGRITY-VALIDATION` (5 invariants: validator present+executable; consumer checkset present; the no-divergence gate wired into each of the three seams; fingerprint sidecar fresh; `selfcheck` golden-good/ golden-bad/negative-control wired into meta-test) + paired §1.1 mutation. **Canonical authority:** constitution submodule `Constitution.md` §11.4.186. Non-compliance is a release blocker. No escape hatch — no `--skip-divergence-gate`, `--audit-after-export-OK`, `--cross-doc-check-not-applicable`, `--unvalidated-analyzer-OK`, `--divergent-commit-OK` flag.

§11.4.212 (2026-07-16) — Main README is the canonical starting-point / entry point for ALL project documentation — no doc may be an orphan unreachable from README (User mandate, 2026-07-16)

Verbatim operator mandate (2026-07-16): "Manual tests catalog must have linking all through the main README as the starting point of this and any other Project documentation! This fact and mandatory rule MUST be added into the constitution."

The main README (`README.md` at the repository root) is the canonical STARTING POINT / entry point for ALL project documentation, for THIS project and EVERY project incorporating this constitution. Every project document in the §11.4.65 export scope — INCLUDING the manual-tests catalog (the §11.4.185 manual-QA testing catalog), the §11.4.153 per-feature Status / Status_Summary docs, the Issues / Fixed trackers + their summaries (§11.4.12 / §11.4.53 / §11.4.57), and every guide / research note / changelog / script companion doc — MUST be reachable / linked (DIRECTLY or TRANSITIVELY) from the main README. No project document may be an ORPHAN — a doc that no link-path from README reaches is a §11.4.212 violation. This is the STRICT GENERALISATION of §11.4.57: §11.4.57 mandated a `Tracked-Items + Status Documents` doc-link section in the README covering the tracker / status doc set; §11.4.212 extends the

same reachability guarantee to EVERY §11.4.65-scope document, so the README is the single navigable root of the project's documentation tree. **(A) Reachability = a link path.** README → guide → sub-doc is fine (transitive); the requirement is that SOME path from README reaches every in-scope doc, not that every doc is linked directly at top level. A doc no path reaches is an orphan. **(B) Always-sync (§11.4.59 / §11.4.106 docs-chain).** The README doc-graph is kept current mechanically — a newly-added §11.4.65-scope doc that no README-reachable path links is a §11.4.212 violation the moment it lands; the manual-tests catalog specifically MUST be linked from the README (the operator's load-bearing example). **(C) Honest boundary (§11.4.6).** Reachability is a link-graph property (every doc is FINDABLE from README), NOT a claim about each doc's CORRECTNESS or freshness (those stay §11.4.44 / §11.4.106 / the doc's own gates). **(D) Composition with §11.4.57.** §11.4.57's `Tracked-Items` section remains the mandated README landing block for the tracker / status set; §11.4.212 adds that EVERY OTHER in-scope doc is likewise reachable — §11.4.57 is the specialisation, §11.4.212 the generalisation.

(E) README INTRODUCTION + ILLUSTRATION (extension, 2026-08-15). The main README MUST open with a SHORT introduction (what the project is, who it is for, its current state at a glance — the reader's 30-second answer) AND MUST ILLUSTRATE all work done: for every major capability the project ships, the README carries either a direct summary paragraph OR a link to a detailed document explaining WHAT was done AND WHY it was done (never orphaned links, never "see the changelog" as the only explanation). Every linked document MUST justify its own existence in one sentence at the link site — a link with no "why this matters" is a §11.4.212(E) violation of §11.4.91 summary-clarity severity at the entry-point layer.

(F) LINKAGE-COMPLETENESS RATCHET (extension, 2026-08-15). The reachability graph from README (transitively per (A)) MUST cover the full §11.4.65-scope doc-set enumeration with a monotone-decreasing UNREACHED count (mirrors the §11.4.135 ratchet pattern); a §11.4.65-scope doc that is not reachable from README at pre-build/release seams FAILs the extended `CM-README-DOC-ENTRYPOINT-COMPLETE` gate; brownfield adoption per operator §11.4.66 (§11.4.224(E) fence). The existing `CM-COVENANT-114-212-PROPAGATION` gate stays unchanged (literal `11.4.212` unchanged, propagation stays GREEN);

`CM-README-DOC-ENTRYPOINT-COMPLETE` is EXTENDED with clauses (E)/(F); a new paired mutation strips the introduction, OR removes a "why this matters" annotation, OR breaks reachability of an in-scope doc → the extended gate FAILs.

Classification: universal (§11.4.17) — the consuming project supplies its README path, its manual-tests-catalog path, and its §11.4.65 doc-set enumeration per §11.4.35. Composes §11.4.57 / §11.4.59 / §11.4.65 / §11.4.106 / §11.4.153 / §11.4.185 / §11.4.44 / §11.4.12 / §11.4.53. Propagation gate `CM-COVENANT-114-212-PROPAGATION` (literal `11.4.212` across the consumer fleet) + recommended gate `CM-README-DOC-ENTRYPOINT-COMPLETE` (every §11.4.65-scope document is reachable — directly or transitively — from the main README; the manual-tests catalog is linked) + paired §1.1 meta-test mutation (orphan an in-scope doc so no README path reaches it → the gate FAILs; strip the literal → the propagation gate FAILs; gate-code = separate work item).

Canonical authority: constitution submodule `Constitution.md` §11.4.212.

Non-compliance is a release blocker. No escape hatch — no `--orphan-doc-OK`, `--readme-not-entrypoint`, `--skip-readme-link`, `--doc-unreachable-OK` flag.

2026-07-17

§11.4.215 — A doc that BINDS work MUST live, tracked, in the repository where that work happens (research-derived, 2026-07-17)

Forensic anchor (genericised, 2026-07-17). In a consuming project, the document that DEFINED a binding acceptance gate — the gate a body of work was ordered to satisfy before proceeding — was **untracked** and lived in an out-of-tree scratch directory belonging to a different working checkout, i.e. it existed in NO repository and in no other agent's or operator's checkout. The gate could therefore not be honoured (agents could not read it), not be reviewed (no diff, no history, no authorship), not be version-controlled (no revision, no rollback, no §11.4.44 header), and not be reconciled when it contradicted the source of truth. A binding artifact that lives nowhere is not governance — it is a rumour with an imperative mood, and work ordered against it cannot be verified against it by anyone but the one process that happened to hold the file.

The mandate. Any document that BINDS work — one that defines a gate or acceptance criterion, states a contract the work is judged against, or carries a plan/order the work is required to satisfy — MUST be tracked, in the repository where that work happens, at a declared path, BEFORE it may be cited as binding. **(1) BINDING ⇒ TRACKED, and un-tracked ⇒ NOT BINDING.** A binding claim sourced from an untracked or out-of-tree document MUST NOT be enforced against any work; the correct response to "the gate is defined in a file nobody can read" is to land the document (or decline the gate), never to obey a citation of it. **(2) IN THE REPOSITORY THE WORK HAPPENS IN.** Tracked in a sibling checkout, a scratch directory, a chat message, an issue comment, or a personal note is NOT tracked for this purpose — the test is whether an agent or operator who clones ONLY the repo the work happens in can read the binding text at its declared path. Where a binding doc must govern several repositories, it is inherited BY REFERENCE from a shared, tracked, version-controlled dependency (§11.4.28 / §11.4.177), never copied and never left out-of-tree. **(3) THIS IS THE PRIOR QUESTION §11.4.212 CANNOT ASK.** §11.4.212 mandates that every doc in the export scope be README-reachable — but its check ENUMERATES the in-repo scope and asserts each member is reachable, so a document that exists in no checkout is never enumerated and the gate cannot FAIL on it. §11.4.212 answers "is it findable?"; §11.4.215 answers "is it here at all?". Once a binding doc is in the repo, §11.4.212 (reachable from the README), §11.4.44 (revision header), §11.4.65/§11.4.73 (exports), and §11.4.106 (kept in sync) all attach to it as normal — this anchor is their precondition, not their competitor. **(4) THE GENERALISATION OF §11.4.95.** §11.4.95 is this exact rule scoped to ONE named artifact: an authoritative tracker database is TRACKED, NEVER gitignored, "NOT a build artefact, it IS authoritative source data", as an explicit named carve-out from the gitignore rule. §11.4.215 widens the artifact class from that one file to **any doc that binds work**, exactly as §11.4.212 was minted as the strict generalisation of §11.4.57. §11.4.11 is the counterweight and stays intact: logs, forensic captures, and scratch artifacts remain out-of-tree and untracked by default — §11.4.215 does NOT track them, and a doc's being untracked is only a defect once it is CITED as binding. That is the discriminator: **binding is what pulls a document into the repository**, and a document nobody cites as binding is governed by §11.4.11, not by this anchor.

Honest boundary (§11.4.6). In-repo presence proves the binding text is AVAILABLE, ATTRIBUTABLE, and VERSION-CONTROLLED — it proves nothing about whether the text is CORRECT, current, or self-consistent (those remain §11.4.44 / §11.4.106 / §11.4.186), and nothing about whether the gate it defines is well-seated (that is §11.4.120's seam-placement clause, whose forensic case was defined by this same out-of-tree document — the two defects were carried by one artifact, which is itself the argument that an unreviewable binding doc is where bad gates hide).

Classification: universal (§11.4.17) — references no project-specific hardware, vendor, or layout; the consuming project supplies its repository, its declared paths for binding docs, and its shared-dependency mechanism per §11.4.35. Composes §11.4.11 / §11.4.28 / §11.4.30 / §11.4.35 / §11.4.44 / §11.4.57 / §11.4.65 / §11.4.73 / §11.4.95 / §11.4.106 / §11.4.120 / §11.4.177 / §11.4.186 / §11.4.197 / §11.4.212. Propagation gate `CM-COVENANT-114-215-PROPAGATION` (literal `11.4.215`) + recommended gate `CM-BINDING-DOC-IN-REPO` (every doc cited as binding by a gate, order, or acceptance contract resolves to a tracked path in the repository the work happens in — or is inherited by reference from a tracked shared dependency; a binding citation resolving to an untracked or out-of-tree path FAILs) + paired §1.1 mutation (move a binding doc out of the tree, or untrack it, while a gate still cites it → the gate FAILs; strip the literal → the propagation gate FAILs; gate-code = separate work item).

Canonical authority: constitution submodule `Constitution.md` §11.4.215.

Non-compliance is a release blocker regardless of context. No escape hatch — no `--binding-doc-out-of-tree`, `--untracked-gate-definition`, `--cite-doc-nobody-can-read`, `--scratch-dir-is-good-enough`, `--binding-by-rumour` flag exists.

§ . . — Comprehensive user-manual + guide + FAQ coverage: every component / service / feature / user-visible workflow ships an operator-usable manual, task-oriented guide, and FAQ, always in sync, no orphan capability (BACKGROUND :: REMINDER :: IMPORTANT operator mandate, - -)

Verbatim operator mandate (2026-08-15, Point 1): *"Whole project and all work done MUST BE fully covered with full user manuals and guides and properly created FAQs."*

Compact summary: every consuming project MUST cover the WHOLE project (every component, service, subsystem, binary, container, plugin, feature, and user-visible workflow) with three cooperating documentation classes — (a) a complete USER MANUAL per component/service/product, (b) task-oriented GUIDES for every user-facing workflow, and (c) properly-created FAQs derived from real operator/QA/end-user questions — always in sync with the shipped code, always exported per §11.4.65, always reachable from README per §11.4.212, always regenerated via Docs-Chain per §11.4.106, and always tracked as workable items when incomplete per §11.4.197; a component / service / feature that ships without its trio of docs (manual + guide + FAQ) is a §11.4.257 violation of §11.4 PASS-bluff severity at the documentation layer (the feature ships to a user who cannot use it because nothing tells them how; the §11.4 anti-bluff covenant extended to documentation completeness). §11.4.257 GENERALISES §11.4.18 (script docs — a single class of executable artifact) and §11.4.153 (per-feature Status set — a status not a manual) into the full three-class documentation floor for every user-facing surface, and BINDS §11.4.212 (README entry point) + §11.4.106 (Docs-Chain sync) + §11.4.65 (four-format export) into the always-in-sync guarantee.

(A) COMPREHENSIVE COVERAGE — no orphan capability. Every component / service / subsystem / binary / plugin / user-visible feature enumerated in the project's shipped surface (the §11.4.153 feature Status roster or its equivalent SSoT) MUST map to at least ONE user manual (component-level operator reference), at least ONE task-oriented guide per user workflow that surface exposes, and one FAQ entry per real recurring question captured from operators /

QA / end-users. A capability the project ships that no manual documents, no guide covers, and no FAQ addresses is UNSHIPPABLE regardless of how well the code works — the user has no path to use it.

(B) THREE DISTINCT DOCUMENT CLASSES, none substitutes another. (a) USER MANUAL — the component's operator reference: what it is, what it does, its inputs/outputs/env vars/config/ports/dependencies, its start/stop/status commands, its logs/metrics/health surface, its failure modes and their remediation; complete enough that an operator can run + supervise + debug it without reading the source. (b) GUIDES — task-oriented, workflow-shaped documents, one per user-facing workflow (setup, configuration, integration, migration, troubleshooting, security-hardening); each guide walks the user through a real journey, with prerequisites, step-by-step commands, expected output at each step, verification checkpoints, and rollback path. (c) FAQs — questions PROPERLY CREATED from real recurring signal (operator questions, QA reports, end-user tickets, agent-found-defect narratives), NEVER fabricated marketing FAQ; every entry cites the question source class + resolution + link to the manual/guide section that owns the durable answer.

(C) ALWAYS IN SYNC. Every doc carries the §11.4.44 revision header; Docs-Chain (§11.4.106) is bound at commit / build / constitution-pull write-seams so an edited component whose docs are stale REFUSES the commit; four-format export per §11.4.65 (.md + .html + .pdf, + .docx where the class warrants — the §11.4.153 exception); every doc reachable transitively from README per §11.4.212. FAQ entries that describe now-fixed defects are RETAINED with a "resolved in vN.N.N" note (they answer the user asking "why did this happen to me"), never silently deleted.

(D) TRACKABLE COMPLETION. Missing docs are §11.4.197 items (research/ kicked-off-work completion mandate) — an undocumented capability is an un-wired requirement whose loss is FORBIDDEN; the workable-items DB (§11.4.93/.95) carries per-surface docs-status rows so "what is documented / what is not" is a live query, never a periodic manual audit.

(E) HONESTY. Where a genuine gap exists (a new capability whose docs are in flight), it is HONESTLY MARKED at both the README entry point + the component's Status.md (§11.4.153) as `[DOC-PENDING: <tracker-id>]` per the §11.4.223 provenance-marker discipline, NEVER silently omitted, NEVER faked

with a placeholder paragraph that says nothing (a fake-doc is a §11.4 PASS-bluff at the documentation layer — an operator reads it, learns nothing, and thinks they read the doc).

Honest boundary (§11.4.6). §11.4.257 mandates the DOCUMENTATION EXISTS, is IN SYNC, and is REACHABLE; it does NOT verify the doc is CORRECT in every semantic detail — that stays §11.4.99 (latest-source cross-reference) + §11.4.194 (exhaustive review including doc content) + §11.4.238 (any operator-found doc gap is a coverage-escape triggering a §11.4.138-family audit). "Comprehensive" is bounded to the shipped-surface enumeration; an internal implementation detail with no user-facing exposure has no user-manual obligation (documentation of it lives under §11.4.18 script/source class or design docs, not this anchor).

Classification: universal (§11.4.17). Composes §11.4.11 / §11.4.18 / §11.4.44 / §11.4.65 / §11.4.86 / §11.4.99 / §11.4.106 / §11.4.148 / §11.4.153 / §11.4.194 / §11.4.197 / §11.4.208 / §11.4.212 / §11.4.223 / §11.4.238. Propagation gate `CM-COVENANT-114-257-PROPAGATION` (literal `11.4.257`) + recommended gates `CM-USER-MANUAL-PER-COMPONENT` + `CM-GUIDE-PER-USER-WORKFLOW` + `CM-FAQ-FROM-REAL-QUESTIONS` + paired §1.1 meta-test mutations.

Canonical authority: constitution submodule `Constitution.md` §11.4.257. Non-compliance is a release blocker. No escape hatch — no `--skip-user-manual`, `--guide-later`, `--faq-optional`, `--capability-without-docs-OK`, `--fake-doc-placeholder-OK`, `--source-of-truth-is-code-comments` flag.

---

§ . . — Architectural + data-flow + state-machine + sequence-diagram coverage: every project ships accurate, exported, machine-derivable-or-honestly-authored diagrams embedded into the docs that describe the system (BACKGROUND :: REMINDER :: IMPORTANT operator mandate, - -)

Verbatim operator mandate (2026-08-15, Point 2): *"Whole project MUST BE fully covered with properly created graphs, diagrams, schemes which will be incorporated into all existing documentation."*

Compact summary: every consuming project MUST cover the WHOLE project with properly created graphs / diagrams / schemes across the FOUR canonical diagram classes — (a) SYSTEM ARCHITECTURE (services, containers, ports, dependencies, deployment topology), (b) DATA FLOW (how information moves between components, storage, external systems), (c) STATE MACHINE (lifecycle of every stateful entity: workable items, orchestrator sessions, long-ops, agents, deploys), (d) SEQUENCE (per critical user workflow and per critical cross-component protocol) — INCORPORATED into the existing documentation surface (README, user manuals per §11.4.257, guides, Status docs per §11.4.153, design docs per §11.4.218/§11.4.219) at the natural point of use, NEVER a lone diagram directory nobody opens; every diagram is (i) exported in a scale-appropriate open format (SVG for structural diagrams; PNG for raster fallback; the source in a diffable text form — Mermaid, PlantUML, D2, or a diffable SVG source — §11.4.220 open-first, never a proprietary-tool-only master), (ii) rendered non-blank + non-degenerate + accurate (the §11.4.107(10) analyzer discipline applied to diagrams — a rendered blank / mislabelled / truncated diagram is a §11.4/§11.4.1 bluff at the visual-doc layer), (iii) reachable from README per §11.4.212, and (iv) in sync via §11.4.106 Docs-Chain so a code change that invalidates a diagram REFUSES the commit until the diagram is updated. §11.4.258 STRENGTHENS §11.4.218 (design-library catalogue — components not architecture) and §11.4.219 (screen catalogues — UI IA not system topology) from design-side coverage into whole-project documentation-side diagram coverage.

(A) FOUR-CLASS COVERAGE. (a) ARCHITECTURE — the deployed / deployable topology: every service, container, binary, port, external dependency, host boundary, network boundary, storage boundary; kept true to the actual `docker-compose.yml` / deployment manifest / running system. (b) DATA FLOW — how information (user request, event, credential, artifact, evidence) traverses the system: source → transformation → sink; each edge annotated with format + protocol + trust boundary. (c) STATE MACHINE — the lifecycle of every stateful entity the system manages: workable items (§11.4.15), long-ops (§11.4.232), orchestrator sessions (§11.4.187), agents (§11.4.147), deploy artifacts (§11.4.108), builds; transitions labelled with the triggering event and the guard that gates it. (d) SEQUENCE — one diagram per critical user workflow and per critical cross-component protocol; actors + messages + return paths + failure branches, not only the happy path.

(B) INCORPORATION — never orphan. Each diagram lives at the point of use (architecture diagram in README + `docs/ARCHITECTURE.md`; per-service data-flow embedded in that service's user manual per §11.4.257; state-machine embedded in the doc that owns the state; sequence diagram in the guide for the workflow it depicts); a `docs/diagrams/` gallery may exist as an index, but the load-bearing appearance is EMBEDDED into the doc where the reader needs it, per §11.4.212 reachability.

(C) OPEN + DIFFABLE. Sources in diffable text (Mermaid / PlantUML / D2 / diffable SVG), NEVER proprietary-tool-only masters (§11.4.220); rendered outputs committed alongside sources so a reader without the render toolchain still sees the diagram; renderers §11.4.201-probed before use, absent-tool ⇒ §11.4.3 SKIP-with-reason not a fake output (§11.4.222 export-wave discipline applied here).

(D) ACCURACY VERIFICATION. Every diagram is rendered non-blank + non-degenerate + accurate: a probe render exists (a Mermaid parse-check; an SVG geometry check; a §11.4.107(10) golden-good vs golden-bad-with-drift fixture for the derived-from-code diagrams); a diagram whose depicted service list, port map, or state set diverges from the SSoT (compose file, workable-items schema, orchestrator registry) is caught by a §11.4.86-fingerprint diff on the SSoT + a stale-diagram audit run at pre-build; the Docs-Chain (§11.4.106) refuses a commit that changes an SSoT without an accompanying diagram refresh where a mapped diagram exists.

(E) DERIVED-WHERE-POSSIBLE. Prefer derived / generated diagrams over hand-drawn where the source data exists: a compose-file-driven architecture diagram, a schema-driven state machine, a route-table-driven sequence; hand-drawn diagrams (for user-journey narratives, conceptual overviews) are LABELED as such with an authored-date and reconciled against the SSoT at each release round.

Honest boundary (§11.4.6). §11.4.258 mandates the diagrams EXIST, are ACCURATE against their SSoT, are EMBEDDED, and are RENDERED HONESTLY; it does NOT claim every possible diagram class is required for every project (an internal library with no runtime deployment has no ARCHITECTURE diagram in the deployment sense — it ships its module-dependency diagram in that class's place). Bounded to the WHOLE-PROJECT coverage the mandate names.

Classification: universal (§11.4.17). Composes §11.4.6 / §11.4.65 / §11.4.86 / §11.4.106 / §11.4.107(10) / §11.4.153 / §11.4.201 / §11.4.212 / §11.4.216 / §11.4.218 / §11.4.219 / §11.4.220 / §11.4.222 / §11.4.223 / §11.4.257. Propagation gate `CM-COVENANT-114-258-PROPAGATION` (literal `11.4.258`) + recommended gates `CM-ARCHITECTURE-DIAGRAM-PRESENT-ACCURATE` + `CM-DATAFLOW-DIAGRAM-PER-SERVICE` + `CM-STATEMACHINE-DIAGRAM-PER-STATEFUL-ENTITY` + `CM-SEQUENCE-DIAGRAM-PER-CRITICAL-WORKFLOW` + `CM-DIAGRAM-EMBEDDED-NOT-ORPHAN` + paired §1.1 mutations.

Canonical authority: constitution submodule `Constitution.md` §11.4.258. Non-compliance is a release blocker. No escape hatch — no `--skip-architecture-diagram`, `--diagram-in-orphan-directory`, `--proprietary-master-only`, `--stale-diagram-OK`, `--dataflow-implied-from-code`, `--sequence-later` flag.

---

§ . . — README quality-status badge row: at the top of every project README, a comprehensive row of quality badges with a closed green→amber→red vocabulary, each backed by machine-derived source data, illustrating quality and production-readiness in one glance (BACKGROUND :: REMINDER :: IMPORTANT operator mandate, - -)

Verbatim operator mandate (2026-08-15, Point 5): *"We MUST ADD at the top of the main README all possible badges with coloring from green (ok) to red (not ok) for all possible areas of work done to the project illustrating quality of work done and how close are we to deploying this project to full production."*

Compact summary: every consuming project MUST render, at the TOP of the main README (immediately below the H1, above the introduction — the reader's first visual signal), a comprehensive BADGE ROW carrying every applicable quality signal with a CLOSED color vocabulary from GREEN (ok) through AMBER (attention) to RED (not ok) — plus a "production-readiness" gauge that summarises how close the project is to full production deployment — with EVERY badge backed by MACHINE-DERIVED source data (never a hand-typed color-of-the-day, §11.4.6 no-guessing at the status-report layer); a project whose README lacks the badge

row, or carries badges whose colors are fabricated / stale / not backed by real source data, is a §11.4.259 violation of §11.4 PASS-bluff severity at the reader-first-impression layer (an operator glances at the README and gets a green picture of a broken project). §11.4.259 EXTENDS §11.4.57 (README doc-link section) with a symmetric BADGE section at the same first-visible-position privilege, and BINDS §11.4.106 (Docs-Chain sync — badges regenerate on every state change) + §11.4.86 (fingerprint-driven sync — badge data derived from the SSoT the badge refers to) + §11.4.201 (guards must assert the real condition — a badge is a guard for the reader's eye).

(A) CLOSED COLOR VOCABULARY. GREEN = healthy at target (all constraints met, no findings); AMBER = attention (metric on-target but degrading, or off-target within acceptable band, or blocked on a tracked follow-up); RED = not ok (metric off-target beyond acceptable band, a §11.4.108 gate failing, a §11.4.135 uncovered guard, an open release-blocker); optional GRAY = N/A (badge class does not apply to this project) with an inline note explaining why. NO other colors, NO other verdicts, NO "yellow" or "orange" or "beta" gradations — the closed vocabulary is the operator's cognitive contract with the badge row.

(B) BADGE CLASS FLOOR — every applicable area covered. The MINIMUM set every consuming project ships (adding project-specific badges as consumer-supplied §11.4.35 DATA is welcome, subtraction is not): (1) BUILD status (last build green/red, artifact fingerprint); (2) TEST-BREADTH per §11.4.27/§11.4.169 (unit/integration/e2e/full-automation/security/performance/anti-bluff — one badge per type, or a rollup with a click-through detail per §11.4.224); (3) CODE COVERAGE per §11.4.224 (measured %, green ≥ target, amber ≥ ratchet-floor, red below); (4) SECURITY POSTURE per §11.4.184 SonarQube scanner + supply-chain integrity per §11.4.246 (reproducible + hermetic builds + SLSA Build Level 2 as fleet-wide minimum); (5) DOCUMENTATION completeness per §11.4.257 (fraction of shipped surfaces with the required manual + guide + FAQ trio); (6) DIAGRAM completeness per §11.4.258 (fraction of shipped surfaces with the required diagram set, non-stale); (7) LIVE HEALTH where applicable (uptime / SLO / recent-crash-rate); (8) OPEN DEFECTS per §11.4.15 (Queued + In-progress + Reopened counts, most-reopened highlighted per §11.4.189); (9) SUPPLY-CHAIN integrity level per §11.4.246 (the SLSA Build Level tracked in `docs/security/SLSA_LEVEL.md`, fleet-wide minimum L2); (10) ZERO-SHORTCOMINGS status per §11.4.261 (audit-

sweep verdict); (11) MACHINE-EVIDENCE COVERAGE per §11.4.262 (fraction of PASS claims with captured evidence paths); (12) PRODUCTION READINESS gauge (see clause (D)).

(C) MACHINE-DERIVED SOURCE DATA. Every badge's color + value is COMPUTED from a live source of truth, NEVER hand-typed. Every badge PROVENANCE — the exact source path/query/build id — is embedded in the badge tooltip OR a companion `docs/BADGES.md` provenance table. A badge whose provenance is `hand-typed` fails §11.4.259.

(D) PRODUCTION-READINESS GAUGE. A composite badge — GREEN when every clause-(B) badge is GREEN AND no §11.4.185 manual-QA blocker is open AND no §11.4.260 production-readiness criterion is unmet; AMBER when one or more badges are AMBER but no RED and no blocker; RED when any badge is RED or any blocker is open. The gauge is the operator's one-glance answer to "can we deploy this to production?" and its color is BINDING on the release decision — a RED gauge is a release-blocker (§11.4.108 layer 4 + §11.4.129 huge-blocker family + §11.4.185 manual-QA + §11.4.236 QA-deploy-readiness).

(E) ALWAYS IN SYNC. Badges regenerate on every state change (build completion, test run, defect open/close, doc change, release cut) via §11.4.106 Docs-Chain at the same write-seams that regenerate the rest of the doc surface; a stale badge (color no longer reflects source data) is a §11.4.229 in-sync violation at the reader-facing layer.

(F) BADGE SELF-VALIDATED. The badge-computer is a guard (§11.4.201): golden-GREEN fixture MUST render GREEN, golden-RED fixture MUST render RED, golden-AMBER MUST render AMBER; a badge-computer that PASSes its golden-RED fixture is the §11.4 bluff itself and a release-blocker. Paired §1.1 mutation flips the golden-RED source data and confirms the badge flips.

Honest boundary (§11.4.6). §11.4.259 mandates every applicable badge is PRESENT + accurately derived; it does NOT claim green badges prove the project is bug-free (badges are necessary-not-sufficient, mirroring the §11.4.224 metric-validation family); a project whose badge row is HONESTLY red is compliant with §11.4.259 (the reader gets the truth) even while it fails other release gates. A badge class that GENUINELY does not apply is GRAY-labeled with a reason, never omitted (silence-as-badge is a §11.4.201(6) FALSE-NULL — the reader assumes green because they see nothing red).

Classification: universal (§11.4.17) — badge PALETTE + AT-TOP-OF-README placement + closed color vocabulary + machine-derived + production-readiness gauge are universal; the SPECIFIC badge palette per project is consumer-owned §11.4.35 DATA. Composes §11.4.6 / §11.4.15 / §11.4.27 / §11.4.44 / §11.4.57 / §11.4.86 / §11.4.93 / §11.4.106 / §11.4.108 / §11.4.129 / §11.4.135 / §11.4.169 / §11.4.184 / §11.4.185 / §11.4.189 / §11.4.201 / §11.4.212 / §11.4.223 / §11.4.224 / §11.4.229 / §11.4.236 / §11.4.238 / §11.4.257 / §11.4.258 / §11.4.260 / §11.4.261 / §11.4.262. Propagation gate `CM-COVENANT-114-259-PROPAGATION` (literal `11.4.259`) + recommended gates `CM-README-BADGE-ROW-AT-TOP` + `CM-BADGE-CLOSED-COLOR-VOCABULARY` + `CM-BADGE-MACHINE-DERIVED-SOURCE` + `CM-PRODUCTION-READINESS-GAUGE` + `CM-BADGE-SELF-VALIDATED` + paired §1.1 mutations.

Canonical authority: constitution submodule `Constitution.md` §11.4.259. Non-compliance is a release blocker. No escape hatch — no `--badges-optional`, `--hand-type-color`, `--skip-production-gauge`, `--badge-not-derived-from-source`, `--silent-omit-red-badge`, `--badge-in-footer-OK` flag.
